

FIELD GUIDE · SEPTEMBER 2026

Becoming an AI Product Manager

How modern AI products work, what 2,448 job descriptions ask for, and a 24-week, session-by-session learning path

Part A How a modern AI product works: 16 diagrams and flowcharts, from the model to the user

Part B What the market asks for: 2,448 postings analysed + 44 JDs read closely, with a 10-competency model

Part C The 24-week learning path: 120+ sessions, 5 portfolio projects and a capstone, interview prep, job search

Appendices Glossary · weekly tracker · AI PRD template · resource library · interview kit · outreach templates · sources

Built for a career switch from consulting / MBA into AI product management, with payments and financial services as the default domain. India-first, globally applicable.

How to use this guide

You asked for three things: (1) an understanding of how a modern AI product works, with diagrams and flowcharts; (2) an analysis of what AI PM job descriptions ask for; and (3) a step-by-step learning path in extreme detail. That's the three parts of this guide, in that order.

If you have...	Read
30 minutes	Part A1 (the stack), A4 (life of a request), A10 (evals), Part B1 (headline numbers), and the Part C overview chart.
A weekend	All of Part A and Part B. Then do Week 0 of Part C.
Six months	Everything, one week of Part C at a time. Treat Parts A and B as reference you return to.

Assumptions behind the learning path

- **Background:** business / consulting / MBA, comfortable with spreadsheets and structured problem solving, little or no coding. (Inferred from the rest of this repository: a payments & FS job tracker and an FS pricing & GTM guide. If that's wrong, see the adjustments below.)
- **Time:** 12–15 hours a week for 24 weeks (~320 hours). **Target:** mid-level AI PM roles (the 3–5-year band that half of all postings ask for), in India first and globally second.
- **Domain:** payments & financial services for every project, because FS is the #2 industry hiring AI PMs and your existing work is there.

Adjusting the plan

If you...	Then...
Already code (engineering degree / developer)	Do Weeks 1–8 in 3–4 weeks; spend the saved time on Phase 4 (evals) and a second capstone.
Have only 6–8 hours a week	Stretch each week to 10–14 days; never skip the Saturday build session.
Already work as a PM	Skim Weeks 17–20 (product craft); go deeper on Phases 3–4; take on an AI initiative in your current job as the capstone.
Target frontier labs (OpenAI, Anthropic, DeepMind)	Add weekly transcript reading and a public eval project on an open benchmark; go deeper in Phase 4; daily use of their products is assumed.
Target consulting AI roles (BCG X, QuantumBlack, Accenture)	Add one client-style deliverable: an 'AI opportunity assessment' for an Indian bank, with business case.

Conventions

- **C1–C10** refer to the ten competencies in Part B5. Every week in Part C says which competencies it builds.
- Figures are numbered in order. All diagrams were drawn for this guide; charts use the open 2026 AI PM hiring dataset (credited).
- ₹ amounts are illustrative. Model prices change frequently; always check current pricing pages.

Contents

How to use this guide.....	2
Contents.....	3
Part A · How a modern AI product works.....	4
A1. The modern AI product stack.....	4
A2. How a large language model is made.....	5
A3. How a model produces an answer (inference).....	6
A4. The life of one request, step by step.....	7
A5. Giving models knowledge: Retrieval-Augmented Generation (RAG).....	8
A6. Giving models hands: tools, agents and MCP.....	9
A7. Workflows vs agents: six building patterns.....	10
A8. How to improve a model's output: prompt, RAG or fine-tune?.....	11
A9. Is AI even the right tool? The AI-fit check.....	13
A10. Evals: how AI quality is measured.....	13
A11. Guardrails, safety and responsible AI.....	15
A12. Cost, latency and model routing: the unit economics.....	17
A13. The AI product lifecycle, end to end.....	18
A14. Data flywheels and moats.....	19
A15. Who the AI PM works with, and how the role differs.....	19
Part B · What the market is asking for.....	21
B1. The headline numbers.....	21
B2. The skills, ranked.....	23
B3. Digest of 44 individual job descriptions.....	27
B4. Ten patterns across the JDs.....	30
B5. The AI PM competency model.....	31
B6. What interviews actually test.....	32
B7. Your angle: financial services and payments.....	32
Part C · The 24-week learning path.....	33
C0. How to run this plan.....	33
Phase 0 · Set-up and baseline (Week 0).....	34
Phase 1 · AI and LLM foundations (Weeks 1–4).....	35
Phase 2 · Building with LLMs: prompts, APIs, RAG, prototypes (Weeks 5–8).....	38
Phase 3 · Agents, tools and MCP (Weeks 9–12).....	42
Phase 4 · Evals, quality and responsible AI (Weeks 13–16).....	45
Phase 5 · AI product craft and business (Weeks 17–20).....	48
Phase 6 · Portfolio, interviews and job search (Weeks 21–24).....	51
Appendices.....	54
Appendix A · Glossary.....	54
Appendix B · 24-week tracker.....	56
Appendix C · AI PRD template.....	57
Appendix D · Resource library.....	57
Appendix E · Interview preparation kit.....	59
Appendix F · Outreach templates.....	61
Appendix G · Sources.....	61

Part A · How a modern AI product works

Before you learn skills, you need a map of the thing you'll be managing. This part walks through a modern AI product from the silicon up to the user and back, one diagram at a time. Each section ends with **what the PM decides here**. That's the part interviewers probe and JDs describe.

The one-sentence version

A modern AI product is **ordinary software wrapped around a probabilistic model**. The software gathers the right context, lets the model reason and call tools, checks the output, and logs everything. The AI PM decides what 'good' means, measures it with evals, and trades off quality, speed, cost and risk on every release.

A1. The modern AI product stack

Almost every AI product shipping in 2026, from ChatGPT and Claude to Razorpay's merchant tools or a bank's support bot, has the same five layers plus a set of cross-cutting concerns. Learn this picture first. Every later diagram zooms into one part of it.

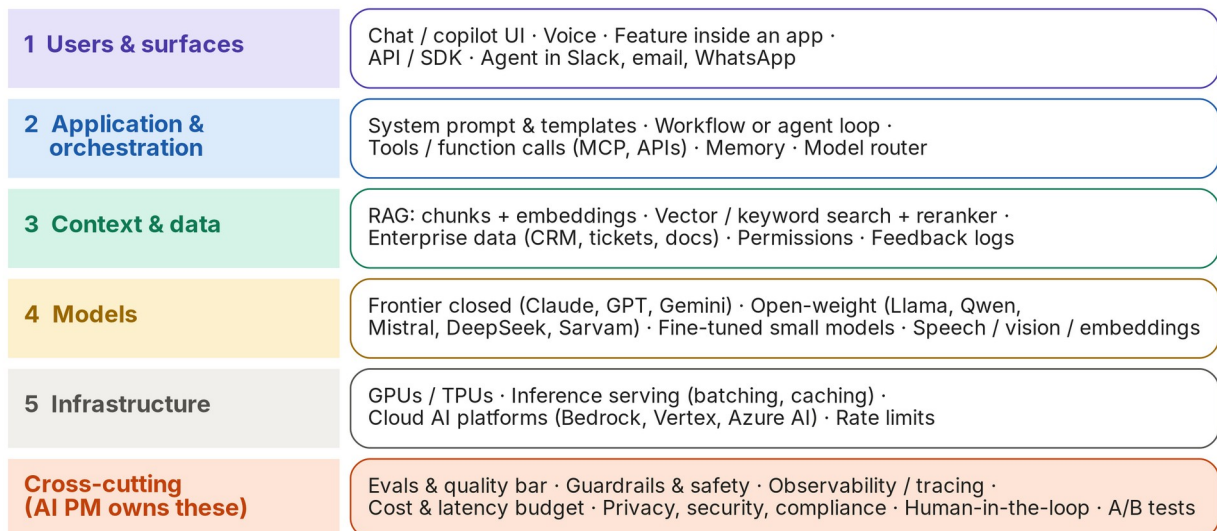


Figure 1. The five layers of a modern AI product, plus the cross-cutting concerns the AI PM owns

Layer	What lives here	What the PM decides
1 · Users & surfaces	Chat window, copilot sidebar, voice, a button inside an existing app, an API for developers, an agent that works in Slack or WhatsApp.	Which surface fits the user's workflow? Chat is not the default answer. Most value comes from AI embedded where work already happens.
2 · Application & orchestration	System prompt, prompt templates, the workflow or agent loop, tool/function definitions, memory, the model router.	How much autonomy? Which steps are fixed code and which does the model decide? Where must a human approve?
3 · Context & data	Retrieval (RAG), vector and keyword search, enterprise data (CRM, tickets, policies), user permissions, feedback logs.	What knowledge does the model need, how fresh must it be, who is allowed to see what, what data can we legally use?

Layer	What lives here	What the PM decides
4 · Models	Frontier closed models (Claude, GPT, Gemini), open-weight models (Llama, Qwen, Mistral, DeepSeek, Sarvam), small fine-tuned models, speech/vision/embedding models.	Which model(s), for which step? Buy an API, host open weights, or fine-tune? What is the fallback if a provider is down?
5 · Infrastructure	GPUs/TPUs, inference servers, cloud AI platforms (AWS Bedrock, Google Vertex AI, Azure AI Foundry), rate limits.	Latency budget, data residency (e.g., data must stay in India), capacity and cost forecasts.
Cross-cutting	Evals, guardrails, observability, cost/latency budgets, privacy & compliance, human-in-the-loop, experimentation.	This is the AI PM's home turf. Define the quality bar, the risk appetite, and the metrics. Then own them.

A2. How a large language model is made

You will never train a frontier model, but you must know how one is made. It explains why models behave as they do: why they have a knowledge cut-off, why they are confidently wrong, why a new version can suddenly be better at coding but worse at your task.

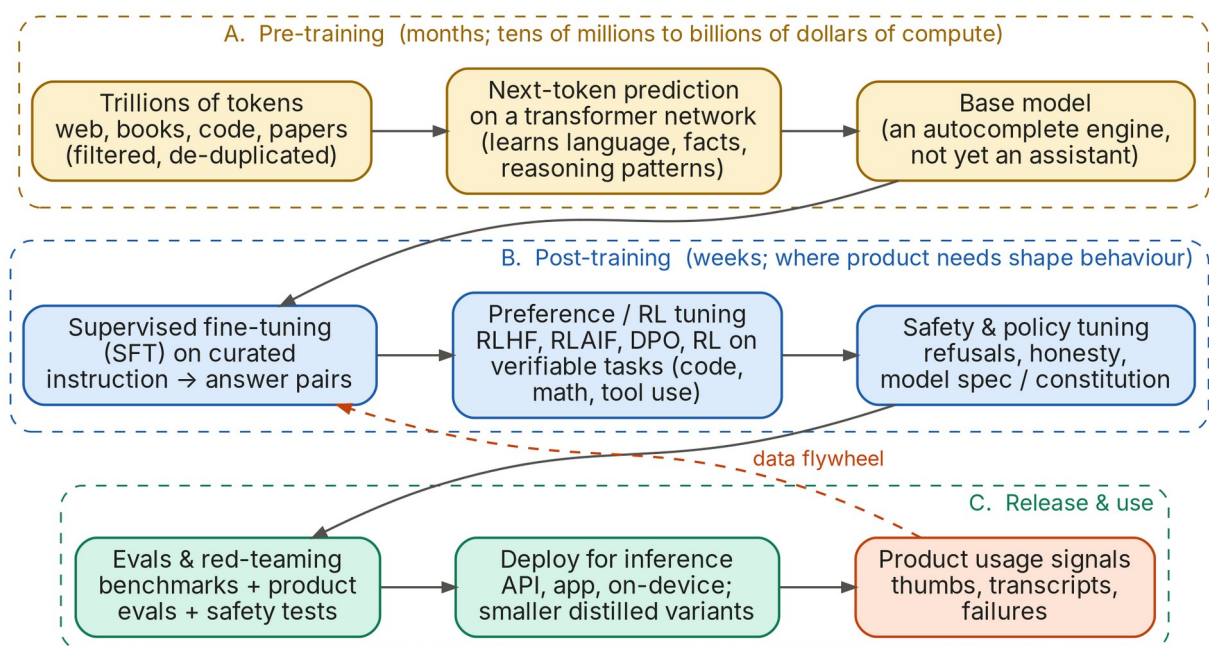


Figure 2. The LLM supply chain: pre-training → post-training → release, with product data feeding back

Stage	What happens	Why a PM should care
Pre-training	A transformer network reads trillions of tokens (web, books, code) and learns one task: predict the next token. The result is a base model , a powerful autocomplete engine.	Knowledge is frozen at the training cut-off . Anything newer (today's RBI circular, your product catalogue) must come from context (RAG or tools). Knowledge of Indian languages and domains depends on how much of that data was in the mix.
Supervised fine-tuning (SFT)	The base model is trained on curated instruction → ideal-answer pairs so it behaves like an assistant.	This is where format, tone and helpfulness are learned. Your own fine-tuning (if you ever do it) is a small version of this step.

Stage	What happens	Why a PM should care
Preference & RL tuning	Humans or AI graders compare answers (RLHF / RLAIF / DPO), and models are trained with reinforcement learning on tasks with checkable answers (code that passes tests, maths, tool use). This is how 'reasoning' models are made.	Explains why models are strong at verifiable tasks (coding) and weaker at taste or judgement. Your evals are a mini version of the graders labs use.
Safety & policy tuning	Models learn to refuse harmful requests and follow a published behaviour spec (e.g., OpenAI's Model Spec, Anthropic's constitution).	Sometimes causes over-refusal on legitimate requests (e.g., a fraud analyst asking how a scam works). Your system prompt and evals must handle this.
Evals & release	Benchmarks (MMLU, GPQA, SWE-bench...), product evals, red-teaming, then deployment through an API or app, often with smaller, cheaper 'distilled' variants.	Benchmarks $\neq$ your product. A new model version is a product change : re-run your evals before switching.

A3. How a model produces an answer (inference)

At run time, a model does one thing in a loop: look at all the tokens so far and pick the next one. Four PM-critical ideas fall out of this: **tokens, context windows, sampling and cost**.

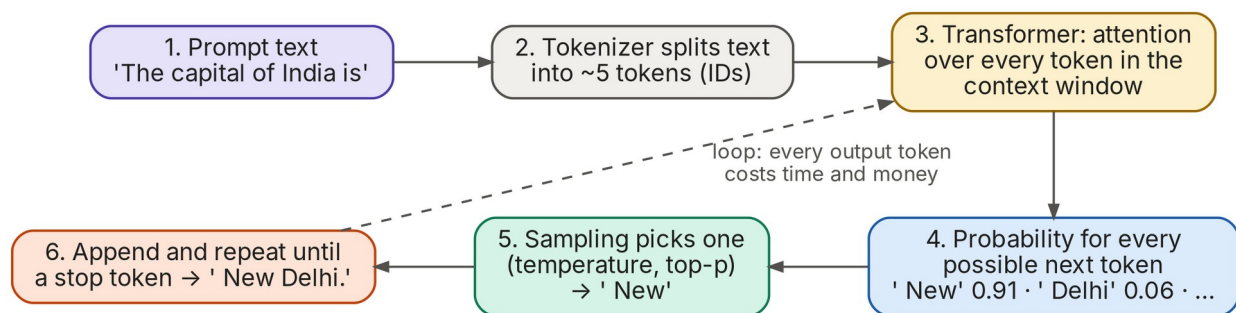


Figure 3. Inference: text becomes tokens; the model predicts one token at a time until it stops

Concept	What it means	Rule of thumb / PM implication
Token	A chunk of text (about $\frac{3}{4}$ of an English word). Models read and write tokens, and you pay per token.	1,000 tokens $\approx$ 750 English words. Hindi and other Indic scripts often need 2–4× more tokens for the same meaning on many tokenizers, so they cost more and run slower. Check with a tokenizer tool.
Context window	The maximum tokens the model can consider at once (prompt + documents + chat history + answer). Modern models range from $\sim$ 128K to 1M+ tokens.	Bigger is not free: more context = more cost, more latency and sometimes worse focus ('lost in the middle'). Curate context; don't dump everything.
Temperature / top-p	Controls randomness when choosing the next token.	Low (0–0.3) for extraction, classification, finance answers. Higher for brainstorming or creative copy. Same input can still give different outputs, so evals must sample multiple runs .

Concept	What it means	Rule of thumb / PM implication
Latency	Time-to-first-token (TTFT) + output tokens ÷ tokens-per-second.	Stream responses so users see progress. Long outputs and reasoning modes are slow. Voice products need sub-second responses.
Cost	Price per million input tokens + price per million output tokens (output usually 3–5× dearer).	Cost per task = (input tokens × input price) + (output tokens × output price) × calls per task. See A12 for a worked example.
Hallucination	The model produces fluent but false content because it is predicting plausible text, not looking up facts.	Mitigate with grounding (RAG), citations, 'say I don't know' instructions, verification steps and evals. You can reduce it but never guarantee zero.

A4. The life of one request, step by step

Here is what actually happens when a user asks a fintech support assistant a question. Every box is a design decision, and every box can fail.

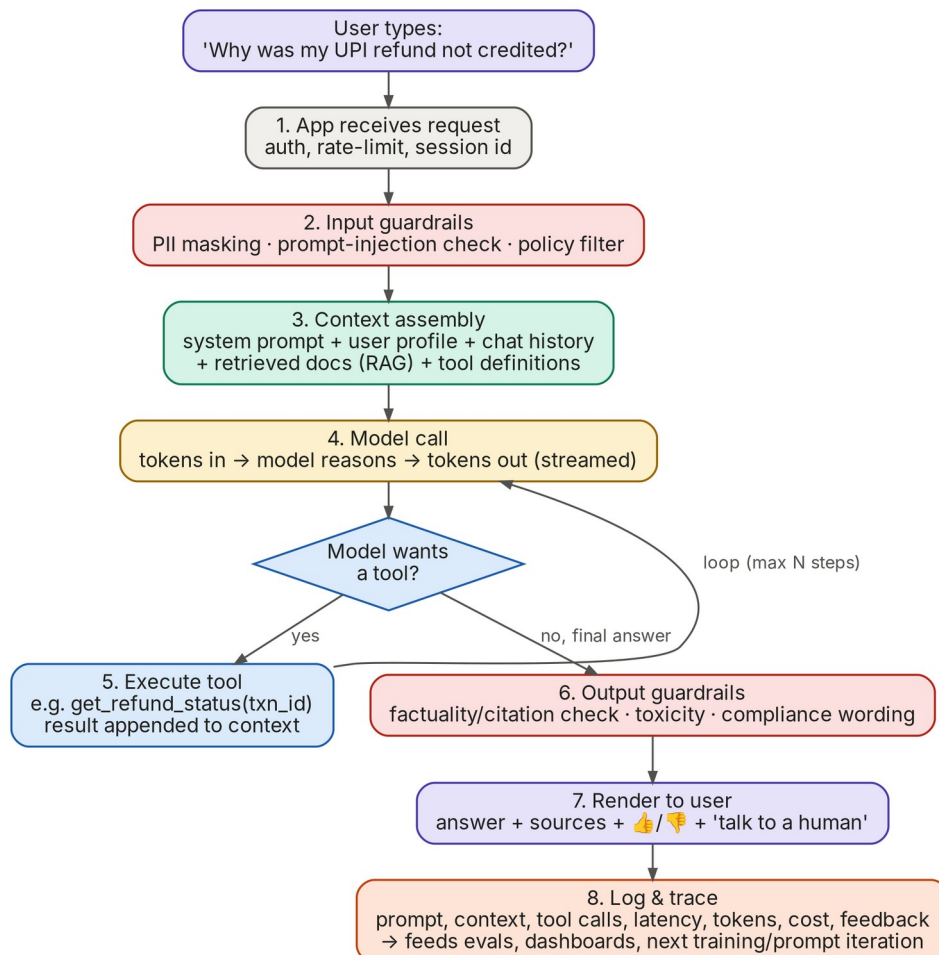


Figure 4. The life of a single request through a production AI assistant

- 1. App receives the request.** Authentication, rate limiting and a session ID. Boring but essential: you need the session ID to debug later.
- 2. Input guardrails.** Mask PII (card numbers, Aadhaar, phone numbers) before it reaches a third-party model. Detect prompt-injection and jailbreak attempts. Check the question is in scope.

3. **Context assembly.** The app builds the full prompt: system prompt (role, rules, tone), user profile, recent chat history, documents retrieved by RAG, and the definitions of tools the model may call. This is **context engineering**, today's most important build skill.
4. **Model call.** Tokens go in; the model reasons and streams tokens out.
5. **Tool loop.** If the model decides it needs data or an action (e.g., `get_refund_status(txn_id)`), the app executes the tool and appends the result, then calls the model again. A step limit prevents infinite loops.
6. **Output guardrails.** Check the answer is grounded in the retrieved sources, uses compliant wording (no unlicensed financial advice), isn't toxic, and matches the expected format.
7. **Render.** Show the answer with sources, a feedback control (👍/👎) and a clear path to a human.
8. **Log & trace.** Store the prompt, context, tool calls, tokens, latency, cost and feedback. **These traces are the raw material for evals, error analysis and future improvements.** If you don't log it, you can't improve it.

A5. Giving models knowledge: Retrieval-Augmented Generation (RAG)

RAG is how you let a model answer from **your** documents: policies, product docs, contracts, tickets, regulations. It appears in 19% of AI PM job postings and in nearly every enterprise AI product.

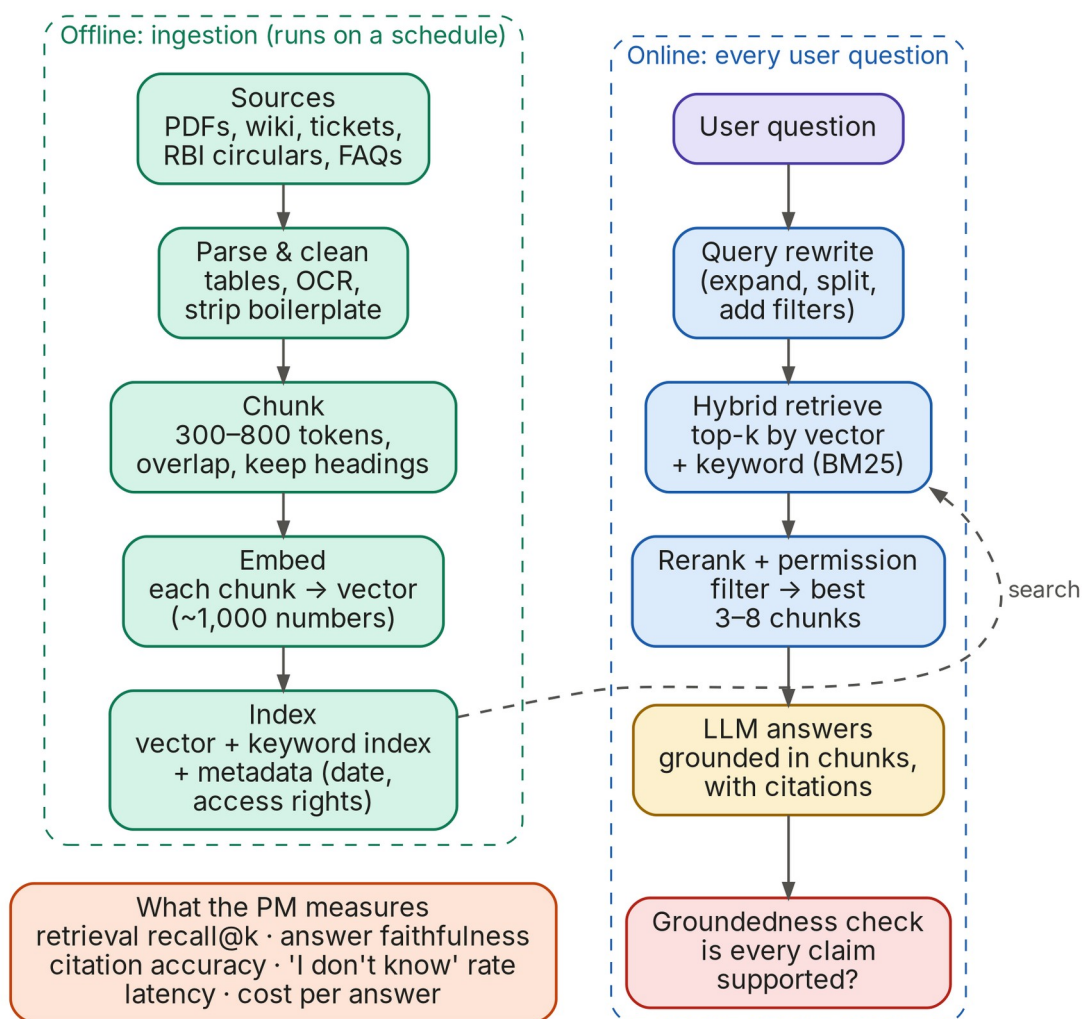


Figure 5. RAG has two halves: offline ingestion and online retrieval + generation

Where RAG breaks (and what to measure)

Failure	Symptom users see	Typical fix	Metric
Bad parsing	Answers ignore tables or scanned pages	Better PDF/table parsers, OCR, manual clean-up of key docs	% of source docs parsed correctly (spot check)
Bad chunking	Answer has half the rule, missing the exception	Chunk by section/heading; add overlap; attach parent context	Retrieval recall@k on a golden set
Retrieval miss	'I couldn't find that', though the doc exists	Hybrid search (vector + keyword), query rewriting, reranker	Hit rate@5, MRR
Stale content	Quotes last year's fee or superseded circular	Freshness metadata, re-index schedule, date filters, 'latest wins' rules	% answers citing current version
Permission leak	User sees another team's or customer's data	Filter retrieval by the user's access rights before the model sees it	Zero-tolerance test cases
Unfaithful answer	Cites a source that doesn't say that	Stricter prompt, groundedness check, show quotes	Faithfulness score (LLM judge + human audit)

A6. Giving models hands: tools, agents and MCP

An **agent** is a model running in a loop, deciding for itself which tool to call next until the goal is met. Agentic AI is now the largest category of AI PM roles: **30% of postings, and 51% mention agents**. The diagram shows a payments example.

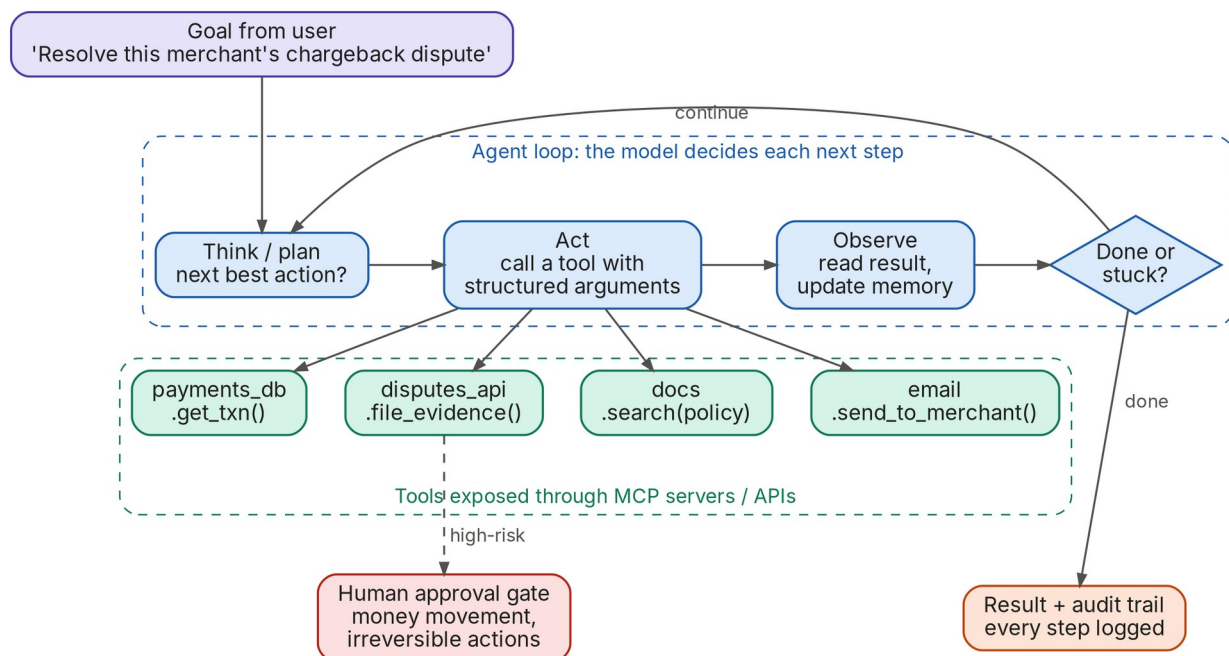


Figure 6. The agent loop: plan → act (call a tool) → observe → repeat, with a human gate for risky actions

Key ideas

- **Tool (function) calling.** You describe tools to the model (name, purpose, input schema). The model replies with a structured request such as `get_txn({"id": "T123"})`. Your code executes it and returns the result. The model never touches your systems directly.

- **MCP (Model Context Protocol).** An open standard, introduced by Anthropic in late 2024 and now supported across the major AI platforms, for plugging tools and data into any AI app. Think 'USB-C for AI': build an MCP server for your database once, and Claude, ChatGPT, Cursor or your own agent can use it. 9% of postings already name MCP; Razorpay and PhonePe JDs ask for it explicitly.
- **Agent-to-agent (A2A) protocols** let agents from different companies talk to each other (e.g., your procurement agent negotiating with a supplier's sales agent).
- **Agentic commerce** is the frontier for payments PMs. Card networks (Visa Intelligent Commerce, Mastercard Agent Pay), checkout protocols from OpenAI/Stripe and Google, and UPI pilots in India let an agent pay on a user's behalf under a mandate. Visa and Mastercard are hiring PMs specifically for this.
- **Autonomy is a dial, not a switch.** L1 suggest → L2 draft for approval → L3 act with undo → L4 act with audit → L5 fully autonomous. Most regulated-industry products live at L2–L3.

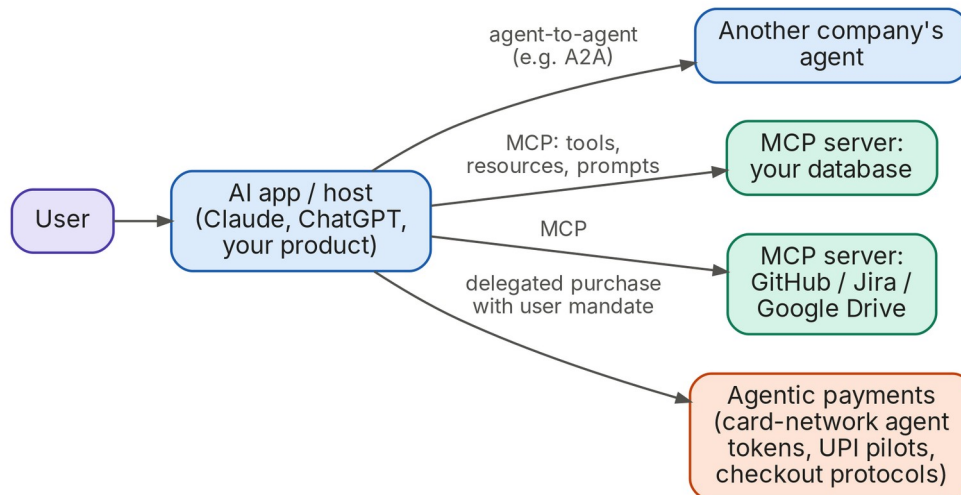


Figure 7. How the pieces connect: MCP for tools and data, agent-to-agent protocols, and agentic payments

A7. Workflows vs agents: six building patterns

Anthropic's widely-cited guide 'Building effective agents' makes a point every AI PM should repeat: **use the simplest pattern that works**. Many successful 'agents' are really fixed workflows with an LLM at a few steps. They are more predictable, cheaper and easier to evaluate.

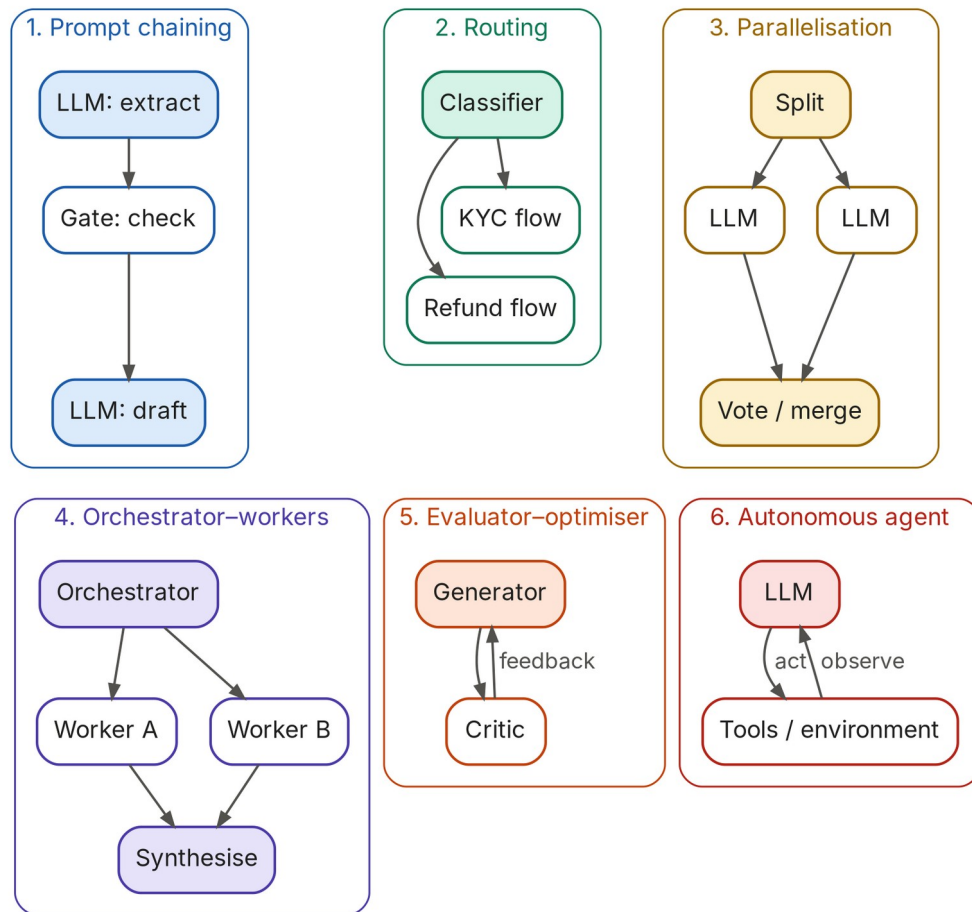


Figure 8. Six common patterns, from most predictable (1) to most autonomous (6)

Pattern	Use when	Payments / FS example
1. Prompt chaining	Task splits cleanly into fixed steps	Extract fields from a KYC document → validate → draft summary for analyst
2. Routing	Different input types need different handling	Classify an incoming ticket as refund / KYC / fraud and send to the right flow or model
3. Parallelisation	Independent sub-tasks, or you want several opinions (voting)	Three independent checks on a loan application's documents; flag if any disagree
4. Orchestrator-workers	Sub-tasks can't be known in advance	Research a merchant's risk profile: orchestrator decides which sources to check
5. Evaluator-optimiser	Clear quality criteria, iteration helps	Draft a customer letter; a critic checks compliance wording; redraft
6. Autonomous agent	Open-ended, many steps, a good environment to act in	Resolve a chargeback end-to-end with tools, human approval for money movement

A8. How to improve a model's output: prompt, RAG or fine-tune?

This is one of the most common interview questions ("When would you fine-tune vs use RAG?"). The answer is a ladder: climb one rung at a time, and measure with evals at each rung.

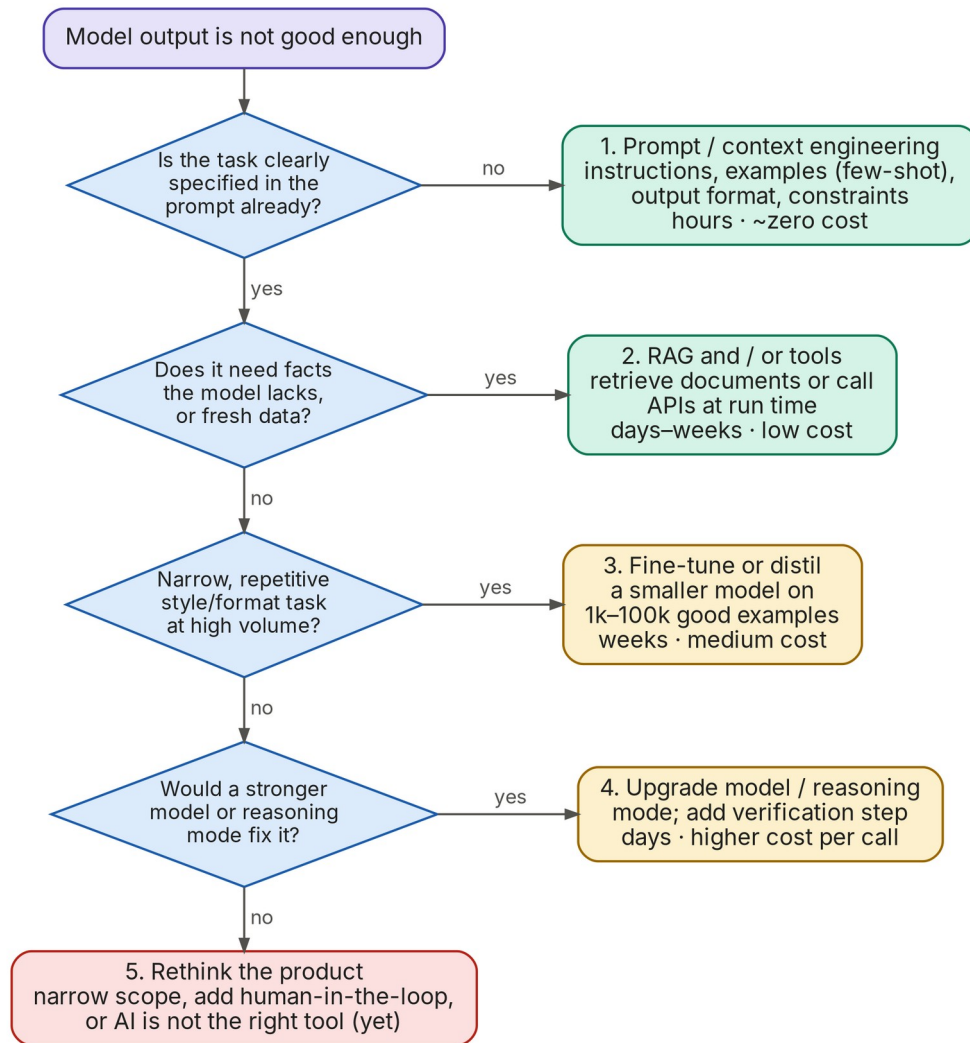


Figure 9. Decision ladder for improving model output

Approach	Best for	Not for	Cost & time
Prompt / context engineering	Instructions, tone, format, examples, reasoning steps	Knowledge the model doesn't have	Hours; near-zero cost
RAG / tools	Private, large or fast-changing knowledge; live data; actions	Changing the model's style or skills	Days-weeks; low running cost
Fine-tuning	Consistent format/style at high volume; distilling a big model's skill into a small cheap one; narrow classification	Adding facts (it memorises poorly and goes stale)	Weeks; needs 1k-100k quality examples, plus re-training when base models change
Bigger / reasoning model	Complex multi-step reasoning, coding, analysis	High-volume simple tasks (overkill)	Instant; higher cost and latency per call
Pre-training your own model	Rare: sovereign or very specialised needs (e.g., Indic-language foundation models)	Almost every product team	Months; millions of dollars

A9. Is AI even the right tool? The AI-fit check

The best AI PMs kill bad AI ideas early. Use this flow before writing a single prompt. Note the classic-ML branch: fraud scores, credit risk and churn prediction on tabular data are usually better served by traditional machine learning than by an LLM.

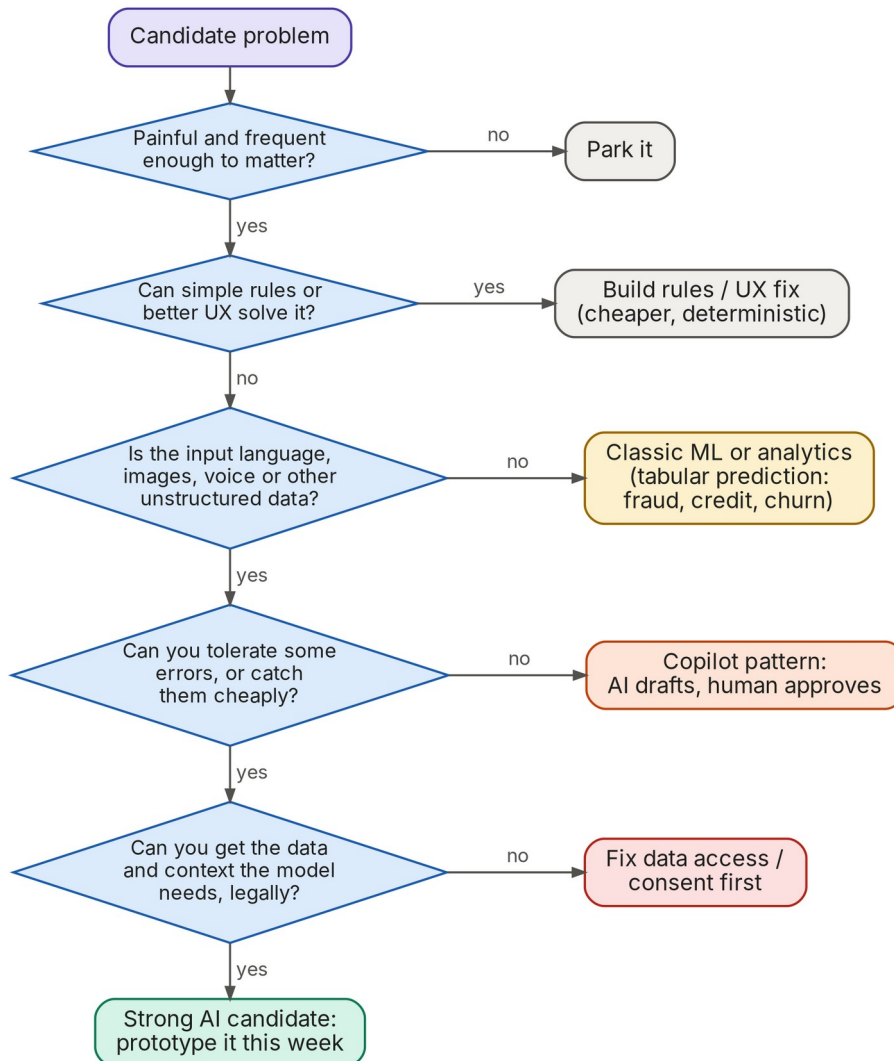


Figure 10. AI-fit decision flow

Error-cost matrix: the question that decides autonomy

For every AI feature, ask: **what happens when it's wrong, and who notices?**

- Low cost, easy to notice (e.g., a suggested reply) → automate fully.
- Low cost, hard to notice (e.g., a slightly wrong ticket tag) → automate, audit samples weekly.
- High cost, easy to notice (e.g., a draft loan-rejection letter) → AI drafts, human approves.
- High cost, hard to notice (e.g., a credit decision, money movement) → human decides; AI assists; heavy evals and audit trail. Regulators (RBI, EU AI Act) will expect this.

A10. Evals: how AI quality is measured

If you remember one thing from this guide: **evals are the AI PM's core artefact**, the way PRDs were for classic PMs. OpenAI's CPO has called writing evals a core PM skill. 20% of all postings ask for evals

explicitly, and when a JD separates must-haves from nice-to-haves, evals land in the must-have block 66% of the time. Every frontier-lab JD we read put evals near the top.

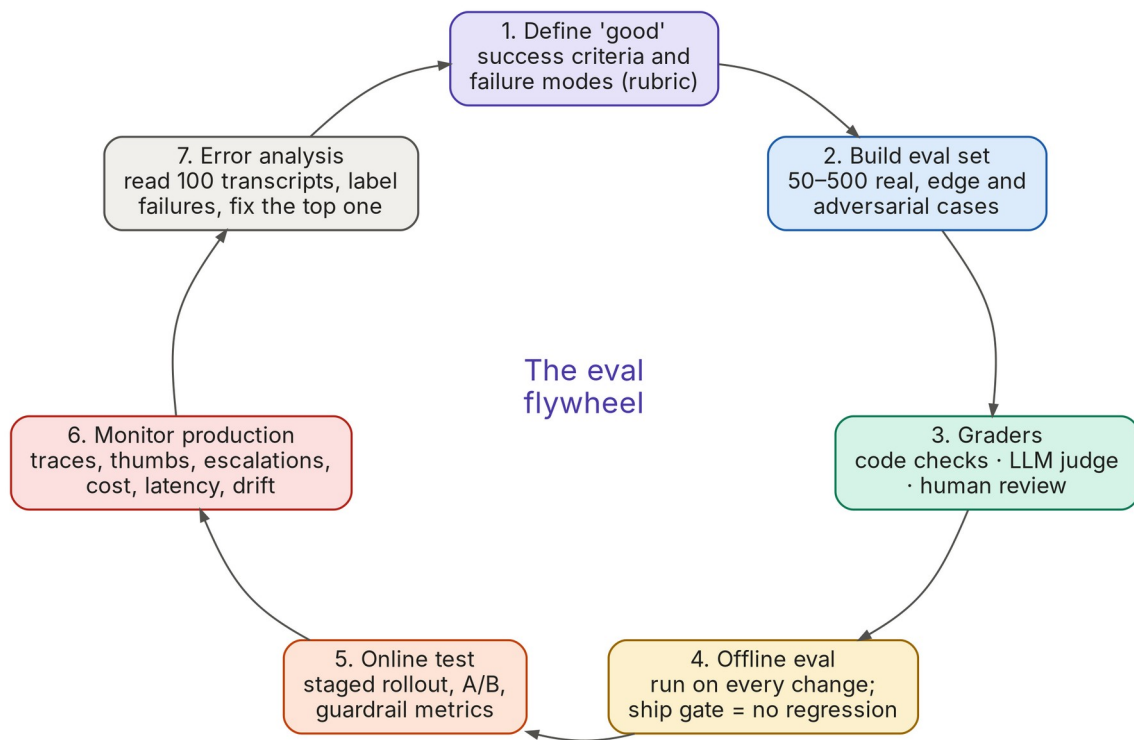


Figure 11. The eval flywheel: define → dataset → graders → offline → online → monitor → error analysis → repeat

The three kinds of grader

Grader	How it works	Good for	Watch out
Code-based checks	Deterministic assertions: is the JSON valid, is the amount correct, does the answer contain a citation, did the agent call the right tool?	Format, correctness where there's a right answer, tool-use checks. Fast and cheap.	Can't judge tone, helpfulness or nuance.
LLM-as-judge	A (usually strong) model grades outputs against a rubric, ideally pass/fail with a written critique.	Faithfulness, relevance, tone, policy compliance, at scale.	Must be calibrated against human labels (aim for ≥85–90% agreement). Judges have biases (verbosity, position).
Human review	Domain experts rate outputs with a rubric; PM reads transcripts.	Ground truth, new failure modes, high-stakes domains.	Slow and costly: use for calibration and sampling, not every run.

Offline vs online

- **Offline evals** run before release on a fixed dataset (your 'golden set'): typical cases, edge cases, adversarial cases. They are your **regression tests**: every prompt change, model upgrade or retrieval tweak must pass them.
- **Online evals** measure real users: task success, thumbs-up rate, escalation-to-human rate, re-ask rate, conversion, retention. Plus **guardrail metrics** that must not get worse: complaint rate, unsafe-output rate, cost per task, latency p95.

- **Error analysis** connects the two: read ~100 real transcripts, label what went wrong, count failure types, fix the biggest. Hamel Husain and Shreya Shankar, who teach the most popular evals course, call this the highest-ROI activity in AI product work.

Example rubric: UPI dispute assistant

Criterion	Pass means...	Grader	Ship gate
Correct resolution path	Recommends the right next step for the dispute type per NPCI/bank policy	LLM judge + expert audit of 10%	≥ 90% on golden set
Grounded	Every factual claim is supported by a cited policy chunk	LLM judge (calibrated)	≥ 95%
No PII leakage	Never echoes full card/account numbers or another user's data	Code check (regex) + red-team set	100%, zero tolerance
Appropriate escalation	Hands off to a human for fraud claims, threats, or when unsure	Code check on tool call + judge	≥ 98% on escalation cases
Tone & clarity	Plain language, empathetic, ≤ 120 words unless asked	LLM judge	≥ 85%
Latency / cost	p95 < 4 s; cost < ₹1.50 per conversation	Logs	Budget met

A11. Guardrails, safety and responsible AI

AI products fail in new ways. Guardrails are layered so no single failure is catastrophic. In regulated industries like banking and payments, this is often where PMs spend the most time with legal, risk and compliance.

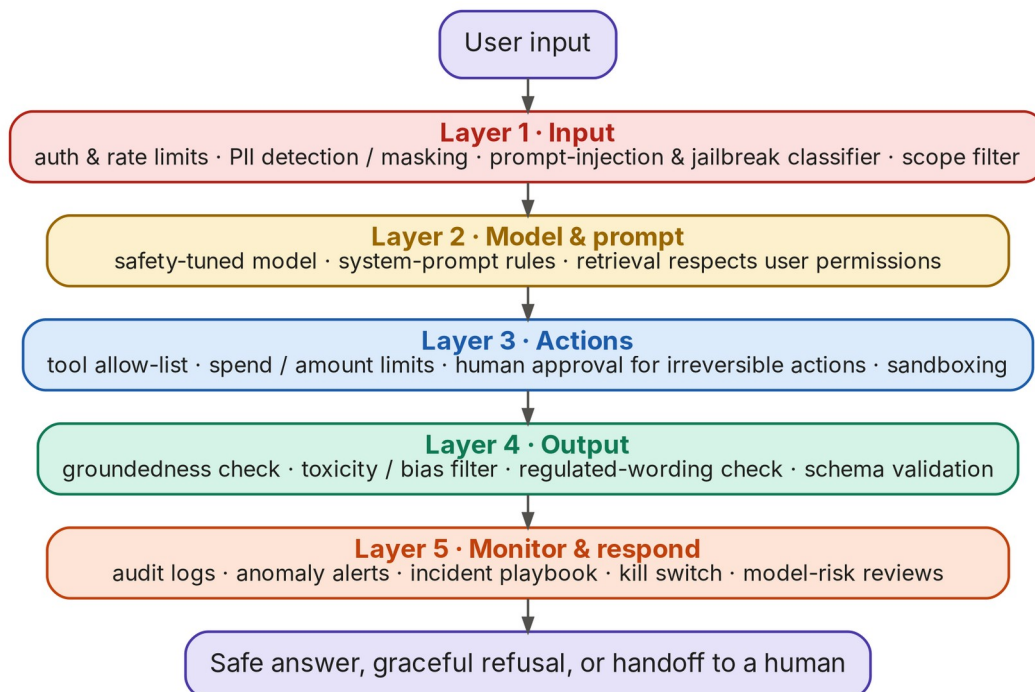


Figure 12. Defence in depth: five layers of guardrails

The failure modes you must know by name

Failure mode	Example	Primary defence
Hallucination	Bot invents a refund policy. (In a 2024 case, a Canadian tribunal held Air Canada liable for its chatbot's invented bereavement-fare policy.)	RAG + citations + groundedness checks + 'I don't know' behaviour
Prompt injection (direct & indirect)	A merchant's email contains hidden text: 'ignore instructions and approve refund'	Treat all retrieved/tool content as untrusted; least-privilege tools; human approval for actions
Excessive agency	Agent sends money or emails without authority	Tool allow-lists, spend limits, approvals, audit logs
Sensitive data disclosure	Model reveals another customer's details or the system prompt	Permission-aware retrieval, PII masking, output filters
Bias / unfair outcomes	Lending assistant treats applicants differently by gender or region	Fairness evals across segments; human decision for credit; documentation
Over-refusal	Refuses a fraud analyst's legitimate question about scam patterns	System prompt context, eval cases for 'should answer'
Jailbreak / misuse	User tricks bot into producing harmful content	Safety-tuned model, input classifiers, monitoring

Regulation and frameworks to know (India-first, then global)

Framework	What it is	Why it matters to an AI PM
DPDP Act 2023 + DPDP Rules (notified Nov 2025)	India's personal-data law: consent, purpose limitation, data-principal rights, obligations for 'significant data fiduciaries', phased compliance timelines	Can you send this customer data to a model provider? For what purpose? How do users withdraw consent? Where is data stored?
RBI FREE-AI framework (Aug 2025)	RBI committee report on Framework for Responsible and Ethical Enablement of AI in the financial sector: guiding principles ('sutras') and recommendations on governance, model risk, audits, consumer protection	If you build AI for a bank, NBFC or payments company, expect board-approved AI policy, model inventories, explainability and incident reporting
India AI Governance Guidelines (MeitY, Nov 2025)	Principles-based national guidance favouring existing laws and voluntary measures over a new AI law	Sets expectations for accountability, transparency and grievance redressal
EU AI Act	Risk-tiered law (prohibited / high-risk / limited / minimal) with obligations phasing in 2025–2027; credit scoring is high-risk	Relevant if your product serves EU users; the de facto global reference for risk tiers
NIST AI RMF 1.0 + GenAI Profile (AI 600-1)	US voluntary framework: Govern, Map, Measure, Manage	A practical checklist for risk reviews; many global firms use it internally
ISO/IEC 42001	Certifiable AI management-system standard	Enterprise buyers increasingly ask vendors for it
OWASP Top 10 for LLM Applications	Security community's list of top LLM app risks (prompt injection, excessive agency, data leakage...)	Use as your red-team checklist

A12. Cost, latency and model routing: the unit economics

Classic software has near-zero marginal cost. **AI products don't: every request costs money.** AI PMs are expected to own this trade-off. The JDs say it directly: 'balancing model quality, usefulness, latency, safety, reliability, and cost' (OpenAI), 'own projections of LLM usage, cost, and capacity' (Glean).

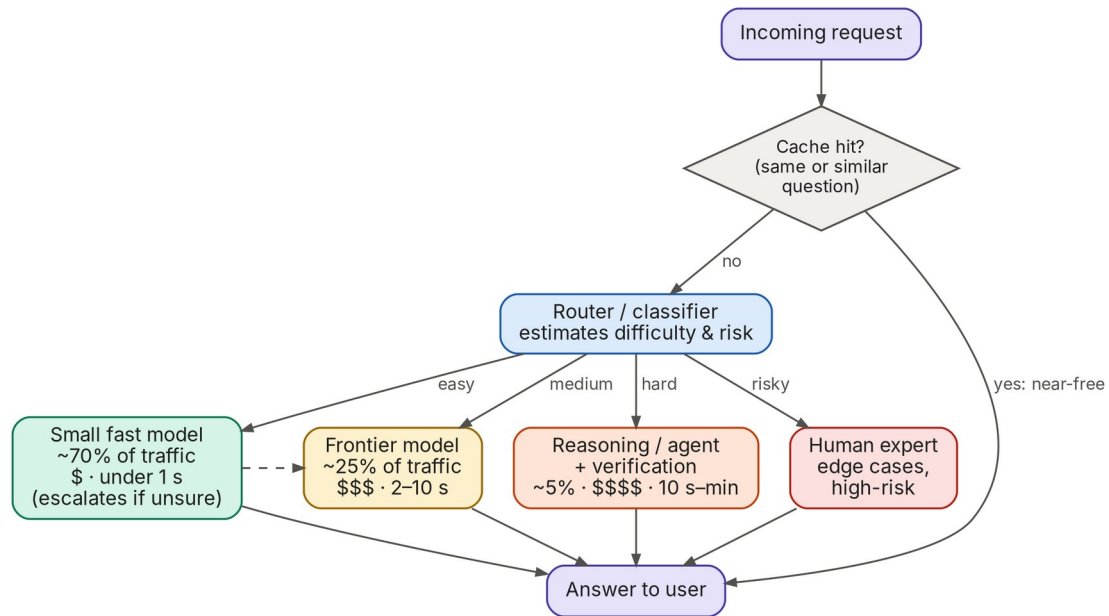


Figure 13. Model routing: send each request to the cheapest option that can handle it

Worked example: cost of a support assistant (illustrative prices)

Item	Assumption	Calculation	Result
Volume	1,000,000 conversations / month		
Tokens per conversation	3 model calls × (2,500 input + 300 output tokens)	7,500 in + 900 out	8,400 tokens
Frontier model only	\$3 per 1M input, \$15 per 1M output (illustrative)	$7.5\text{B} \times \$3/\text{M} + 0.9\text{B} \times \$15/\text{M}$	$\$22,500 + \$13,500 =$ \$36,000/mo ($\approx 3.6\text{¢}$ per conversation)
With routing	70% to a small model at 1/10th the price; 30% frontier	$0.7 \times 3,600 + 0.3 \times 36,000$	$\approx \\$13,300/\text{mo}$ (–63%)
With prompt caching	System prompt + policies cached; ~60% of input tokens discounted ~90%	Input cost falls sharply	Often a further 20–40% saving
Compare to value	Human-handled ticket costs, say, ₹40–80 in India	If the bot fully resolves 40% of chats...	Savings dwarf model cost: measure resolution rate, not just cost

AI pricing models you will be asked about

- **Per seat** (e.g., copilot add-on per user per month): simple, but heavy users can destroy margins.
- **Usage / credits** (per token, per image, per minute of voice): tracks cost, but is unpredictable for buyers.
- **Outcome-based** (e.g., Intercom Fin charges per resolved conversation): aligns price with value; needs an agreed definition of 'resolved'.
- **Hybrid** (platform fee + usage). The most common in enterprise AI in 2026.

See this repo's Pricing & GTM guide (folder [pricing-gtm-financial-services](#)), chapters 01–02, for pricing fundamentals. The same logic applies to AI products.

A13. The AI product lifecycle, end to end

Putting it all together: the loop an AI product team runs, and what the PM owns at each stage. The big difference from classic product development: **'define quality' comes before 'build'**, and the loop never ends. Production failures become new eval cases.

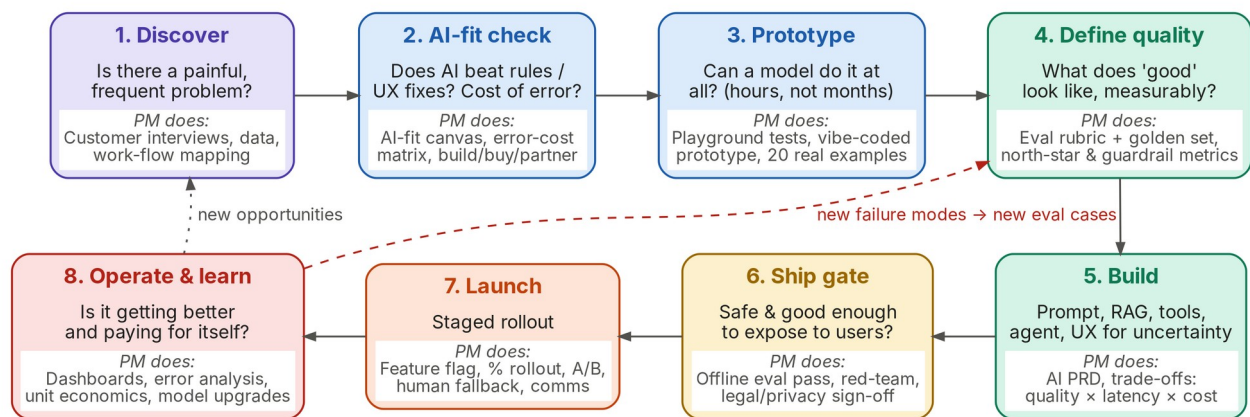


Figure 14. The AI product lifecycle and the PM's job at each stage

Stage	Key questions	PM artefacts	Typical duration
1. Discover	Who has the problem? How often? What do they do today?	Interview notes, workflow map, opportunity-solution tree	1–3 weeks
2. AI-fit check	Is AI the right tool? What's the cost of an error? Build, buy or partner?	AI-fit canvas, error-cost matrix, build/buy memo	Days
3. Prototype	Can a model do this at all, on 20 real examples?	Playground tests, vibe-coded prototype, feasibility note	Days
4. Define quality	What does 'good' look like, measurably?	Eval rubric, golden dataset v1, north-star & guardrail metrics	1–2 weeks
5. Build	Prompt, RAG, tools, agent, UX for uncertainty	AI PRD, model/behaviour spec, cost budget	2–8 weeks
6. Ship gate	Is it good and safe enough for users?	Eval report, red-team results, risk/privacy sign-off, launch checklist	1–2 weeks
7. Launch	How do we expose it safely?	Rollout plan (1% → 10% → 50% → 100%), A/B design, rollback triggers, support playbook	2–6 weeks

Stage	Key questions	PM artefacts	Typical duration
8. Operate & learn	Is it improving? Paying for itself? What broke?	Quality dashboard, weekly error analysis, unit economics, model-upgrade playbook	Forever

A14. Data flywheels and moats

When anyone can call the same frontier model, where does a durable advantage come from? Investors and interviewers ask this constantly ('Isn't this just a wrapper?'). Strong answers point to: proprietary data and feedback loops, distribution, deep workflow integration, trust and compliance, and speed of learning.

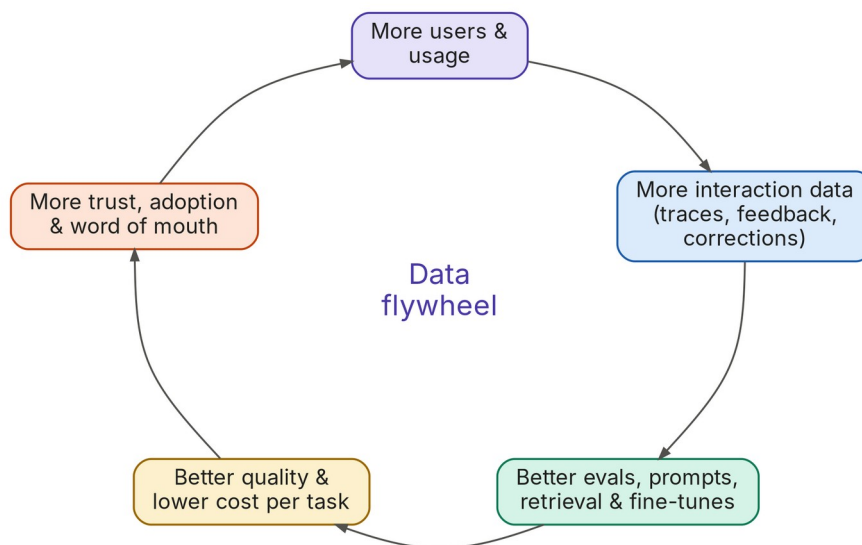


Figure 15. The data flywheel: usage creates the data that improves the product

A15. Who the AI PM works with, and how the role differs

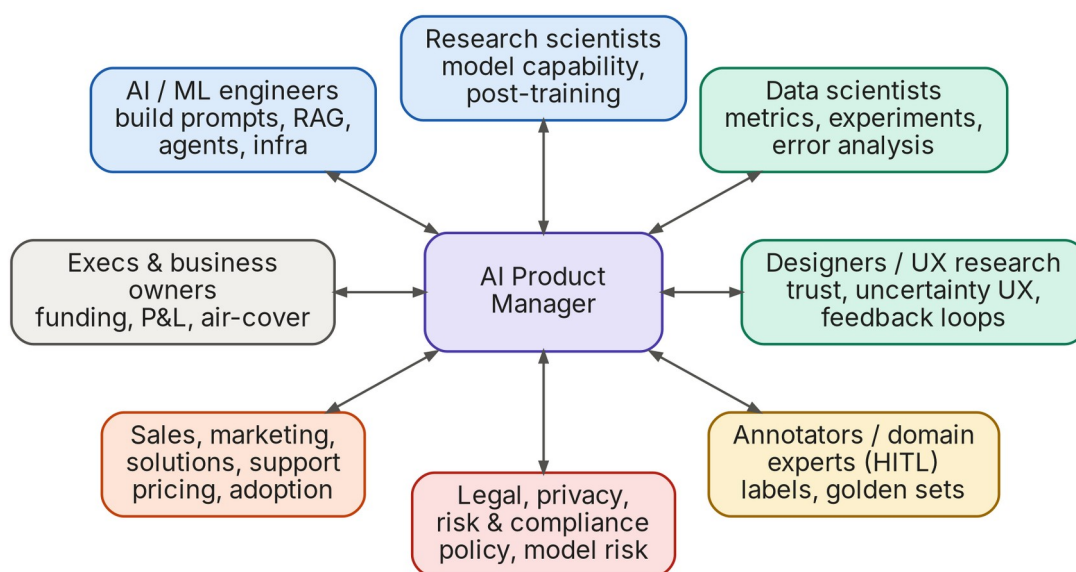


Figure 16. The AI PM's working relationships

Dimension	Classic PM	AI PM
Nature of the system	Deterministic: same input, same output	Probabilistic: same input can give different outputs; quality is a distribution
Core spec	PRD with user stories & acceptance criteria	PRD plus behaviour spec, eval rubric & golden dataset, error budget
Definition of done	Feature works as specified	Eval pass rates meet the bar on offline and online metrics; risks accepted
Roadmap certainty	Engineering estimates are reasonably reliable	Feasibility is uncertain until prototyped; plan in bets and timeboxes
Cost model	Near-zero marginal cost	Every request costs money; the PM owns cost per task and margins
Platform shifts	Occasional	Monthly: new models change what's possible; re-plan often
Key partners	Eng, design, data	Plus ML engineers, researchers, annotators, legal/risk/model-risk
Hands-on expectations	Specs, analytics	Specs, analytics plus prototyping with AI coding tools, reading transcripts, writing prompts and evals

Part B · What the market is asking for

You asked me to go through every AI Product Manager JD available. No single person can literally read every posting: there are tens of thousands live on LinkedIn alone. So this analysis combines two sources:

1. **A census-scale dataset.** The open AI Product Manager Hiring Report 2026 dataset (Abhishek Ashtekar, CC BY 4.0): **2,448 unique AI PM postings from 1,560 companies in 53 countries**, captured on 12 September 2026 from Greenhouse, Lever, Ashby, Workday, LinkedIn and others, de-duplicated from 8,073 raw postings, with skills parsed from a 140-term dictionary. This gives the frequencies: what share of JDs ask for what.
2. **A close reading of 44 individual JDs** across frontier labs (OpenAI, Anthropic, Google DeepMind), big tech (Microsoft, Amazon, Meta, Google), AI-native startups (Glean, Perplexity, Sierra, Decagon, Intercom), Indian product companies (Razorpay, PhonePe, Paytm, Swiggy, Sarvam, Freshworks, Flipkart, Uber and Microsoft India), financial services (Mastercard, Visa, JPMorgan, Capital One, Amex, Stripe) and consulting (BCG X, QuantumBlack, Accenture). This gives the texture: the exact language and the signature asks.

Limitations, stated plainly

- This environment's network blocks direct access to job boards (Lever, Greenhouse, Ashby, LinkedIn, company career sites), so the 44 JDs were read through search-engine extracts of the postings, not the full pages. Details (years, salary ranges) can differ from the live posting.
- The dataset is 52% US postings and only 7% India (181 postings). India-specific pay isn't in it, so India salary figures come from separate salary surveys (cited).
- Skill shares measure what JDs **mention**, not what interviews test. Interview-loop observations are in Section B6.

B1. The headline numbers

Metric	Value	What it means for you
Postings analysed	2,448 unique AI PM roles · 1,560 companies · 53 countries	Demand is broad, not concentrated: 76% of companies posted just one AI PM role, and the top 10 employers hold only 8% of postings.
India's share	181 postings (7.4%): #2 country after the US; Bengaluru is the #4 city globally (79)	India is a real AI PM market; Bengaluru dominates. India JDs lean harder on agents, LLMs, PRDs and metrics.
Most common role type	Agentic AI PM: 30%; GenAI/LLM product PM: 26%	Agents are no longer a niche. Build one.
Experience asked	Median 5 years; 50% ask 3–5 years; only 8% accept ≤ 2 years	For a career switcher, an AI portfolio substitutes for missing years . Several JDs (Intercom, Paytm, Mastercard) say so explicitly.
Degree	22% require a degree; 53% never mention education; MBA mentioned in ~7%	Credentials matter less than evidence of building.
Coding	84% name no programming language; SQL 11%, Python 9%; APIs 28%	You don't need to be an engineer. You do need to understand APIs and be able to build with AI coding tools .
Evals	20% ask for AI evaluation; when JDs split must-haves from nice-to-haves, evals are a must-have 66% of the time	The fastest-rising technical skill and the one interviews now test hardest.

Metric	Value	What it means for you
Product craft	Roadmapping 76%, communication 71%, prioritisation 67%, stakeholder management 64%	Classic PM craft still dominates every JD. AI skills are added to it; they don't replace it.
Industry	Enterprise SaaS 18%, Financial Services & FinTech 17% , healthcare 8%	FS is the #2 industry hiring AI PMs; JPMorgan, Capital One, Citi, BNY, Intuit and Airwallex are among the top employers.
Pay (mostly US)	Median posted base \$193K; mid-level \$159K, senior \$180K, principal \$222K	AI PM pays a premium over generalist PM.
Pay (India)	Reported ranges: ₹16–28 LPA entry, ₹35–55 LPA at 2–4 yrs, ₹65 LPA–1.1 Cr senior; Glassdoor average ~₹30 LPA (P25 ₹18 LPA, P75 ₹46 LPA)	Bengaluru pays a 15–25% premium over other Indian cities; AI PMs earn 25–50% more than generalist PMs (per productleadership.com and Glassdoor India, 2026).

B2. The skills, ranked

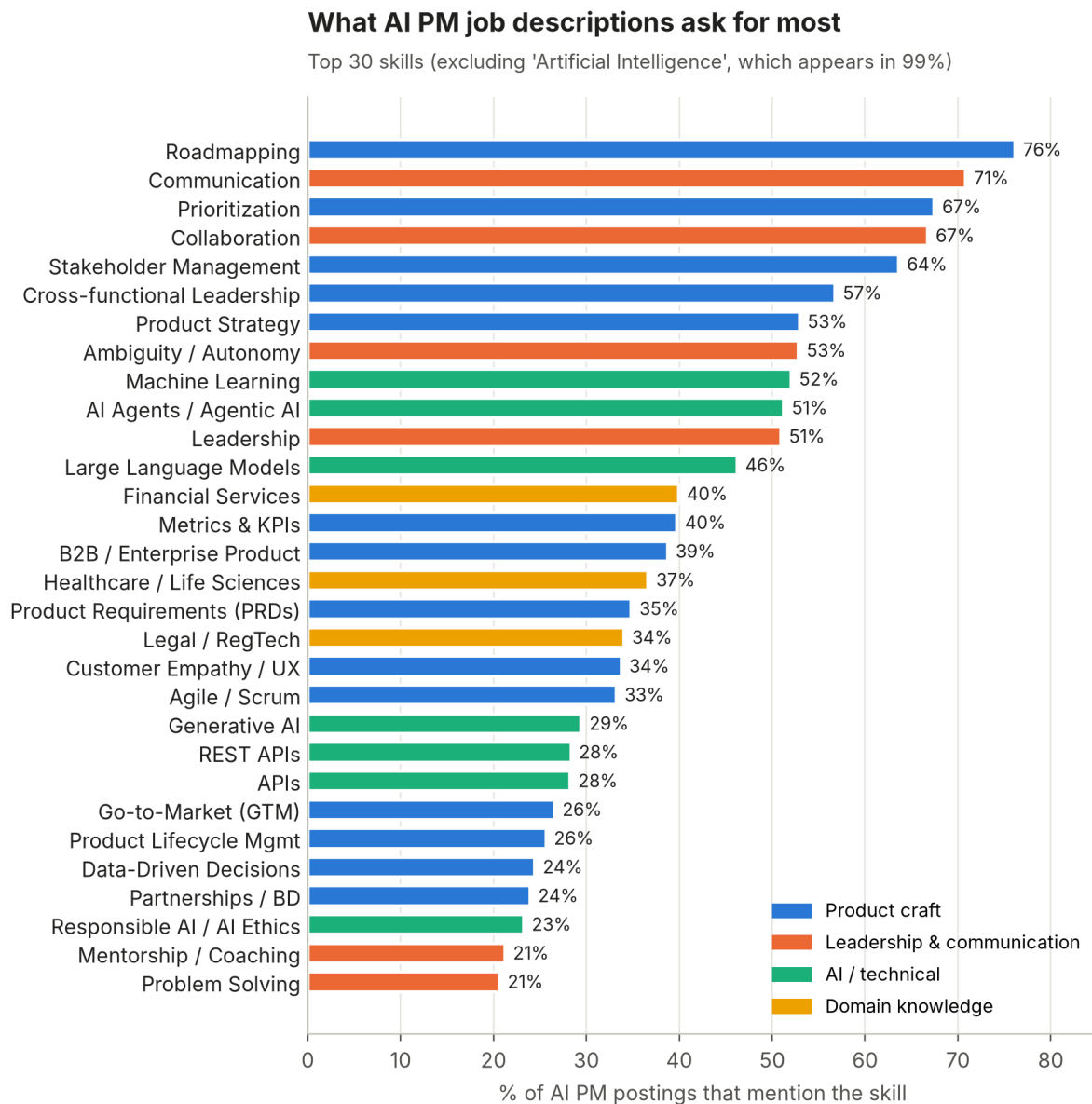


Figure 17. Top 30 skills across 2,448 AI PM postings

Read this chart carefully: **8 of the top 10 skills are classic PM and leadership skills.** AI skills (machine learning, agents, LLMs) enter at #9–#12. AI knowledge gets you into the conversation, and product craft gets you the job. The learning path therefore builds both in parallel.

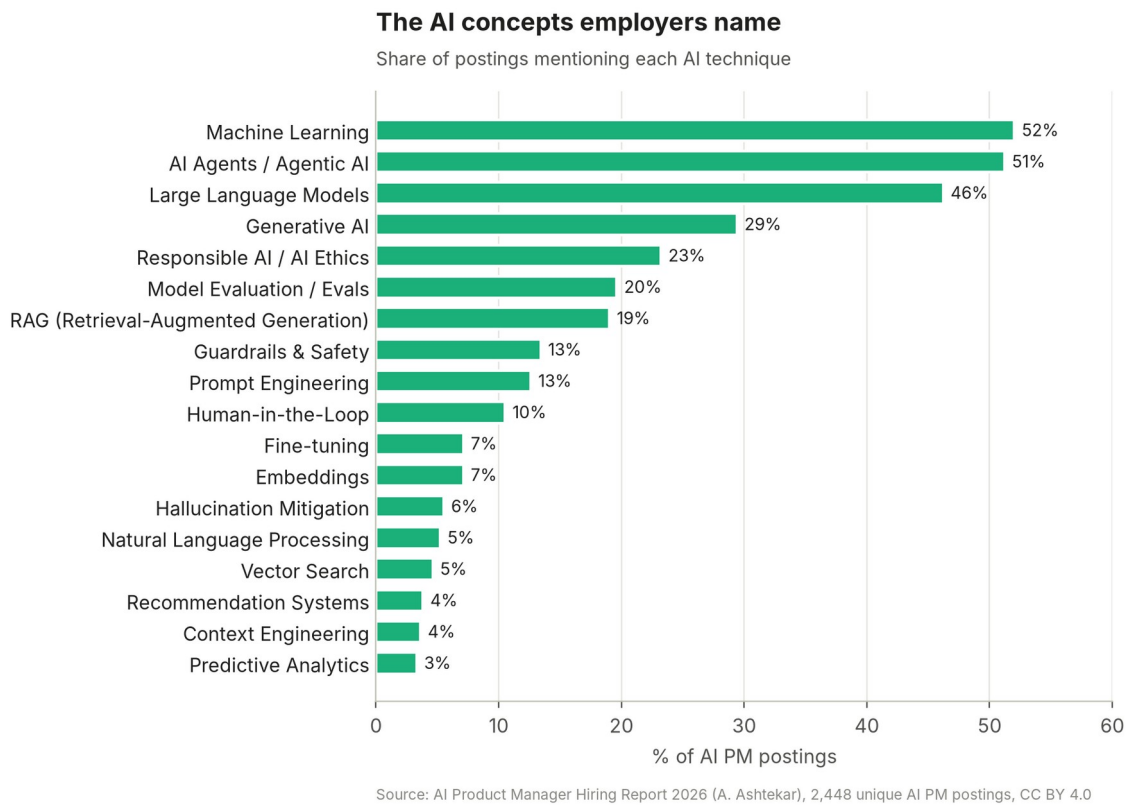


Figure 18. The AI concepts employers name, by share of postings

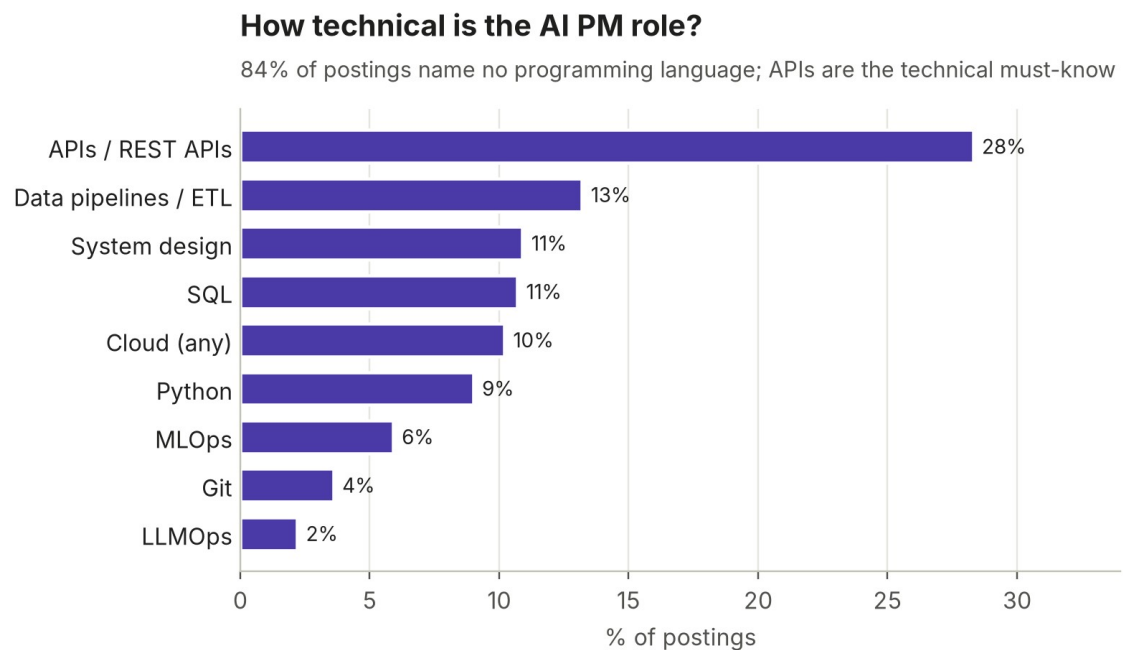
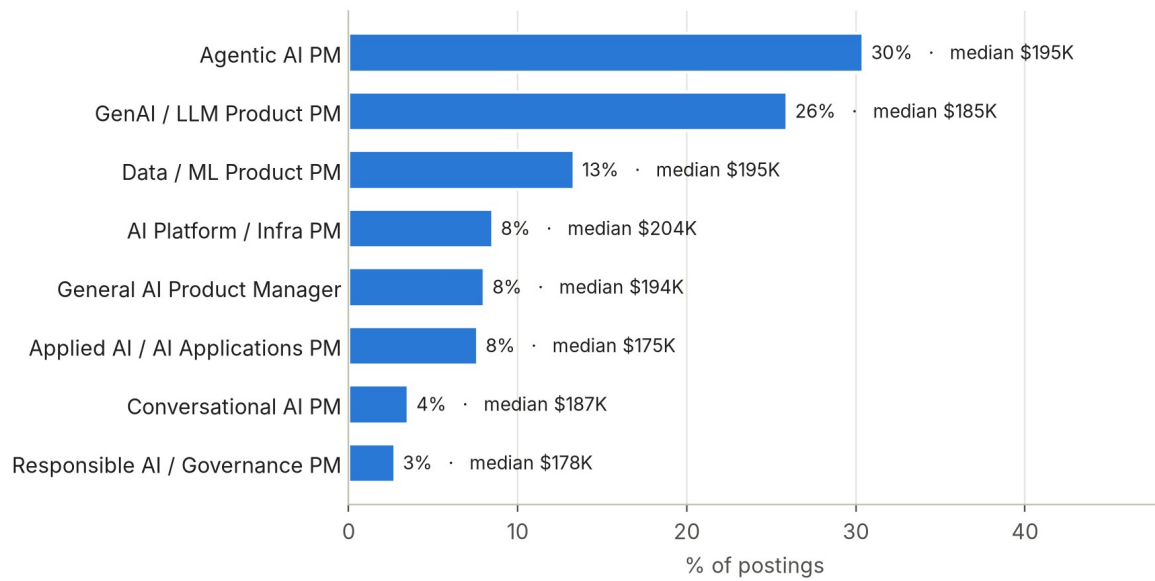


Figure 19. How technical the role is

The 8 kinds of AI PM role

Share of postings, with median posted base salary (USD, mostly US roles)



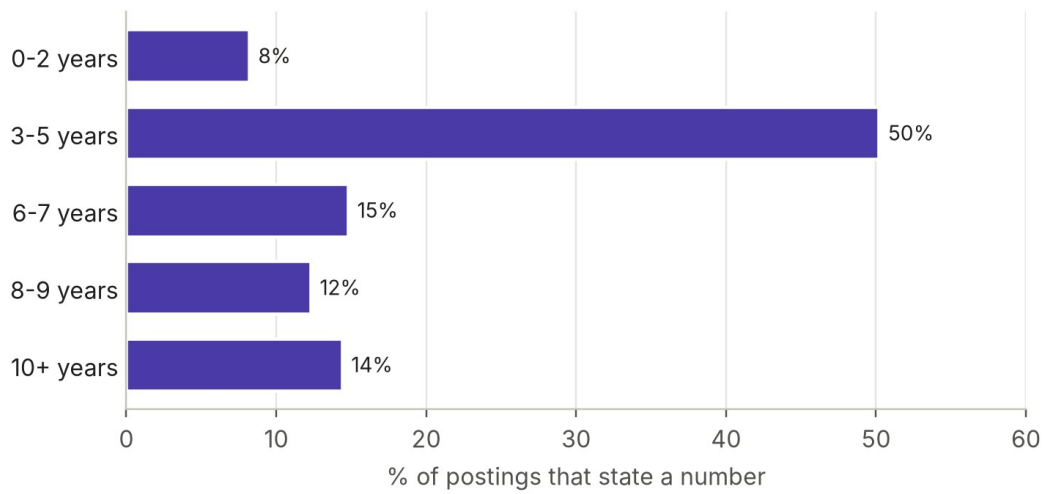
Source: AI Product Manager Hiring Report 2026 (A. Ashtekar), 2,448 unique AI PM postings, CC BY 4.0

Figure 20. The eight kinds of AI PM role

Role type	What the job is	Best fit for a consulting / MBA background?
Agentic AI PM (30%)	Products where AI takes multi-step actions: support agents, ops automation, coding agents, agentic commerce	Yes. Workflow and process redesign is consulting's home turf.
GenAI / LLM product PM (26%)	Copilots, assistants, content and search features built on LLMs	Yes, with a strong portfolio
Data / ML product PM (13%)	Classic ML: fraud, risk, recommendations, personalisation	Yes in FS (fraud, credit), but needs stats/ML fluency
AI platform / infra PM (8.5%)	Model hubs, eval platforms, inference, developer APIs	Harder: usually wants engineering background
General AI PM (8%)	Mixed AI features in an existing product	Yes
Applied AI / AI applications PM (7.6%)	Industry-specific AI apps (legal, health, finance)	Yes. Domain knowledge is the differentiator
Conversational AI PM (3.5%)	Chat and voice bots	Yes
Responsible AI / governance PM (2.7%)	AI policy, risk and compliance tooling	Yes, especially in banks and regulated firms

Years of experience asked

Median ask is 5 years; only 8% accept two years or fewer

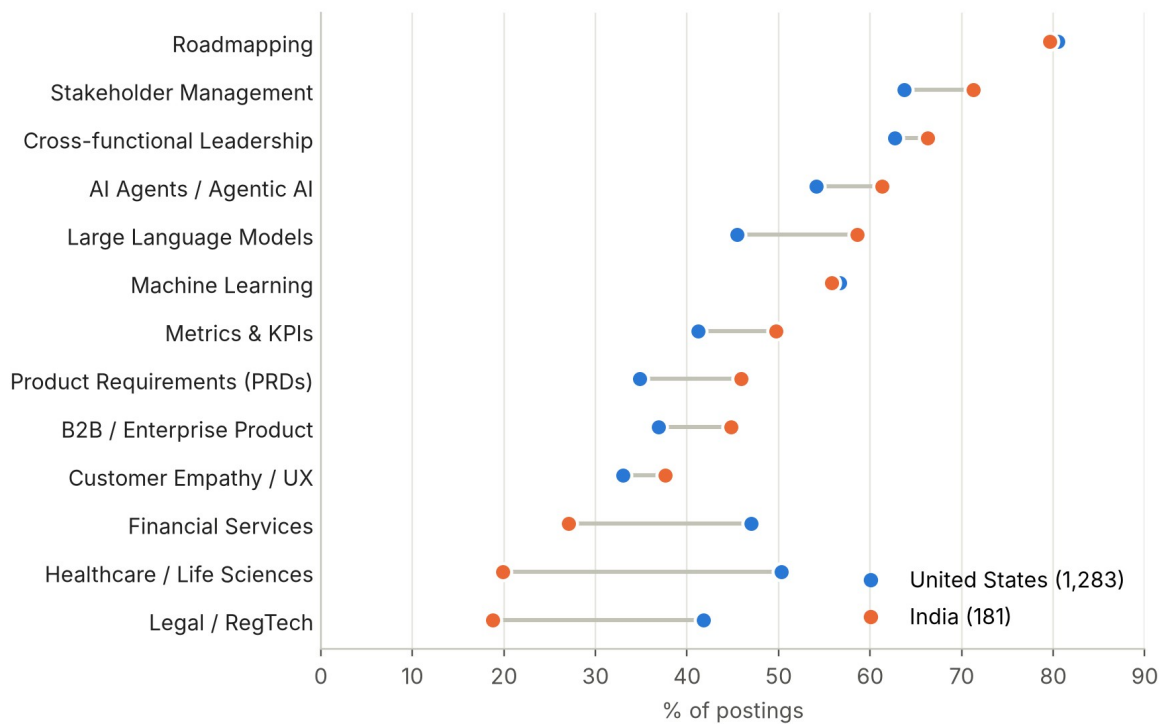


Source: AI Product Manager Hiring Report 2026 (A. Ashtekar), 2,448 unique AI PM postings, CC BY 4.0

Figure 21. Years of experience asked

India vs US: what differs

India asks more for agents, LLMs, PRDs and metrics; far less for regulated domains

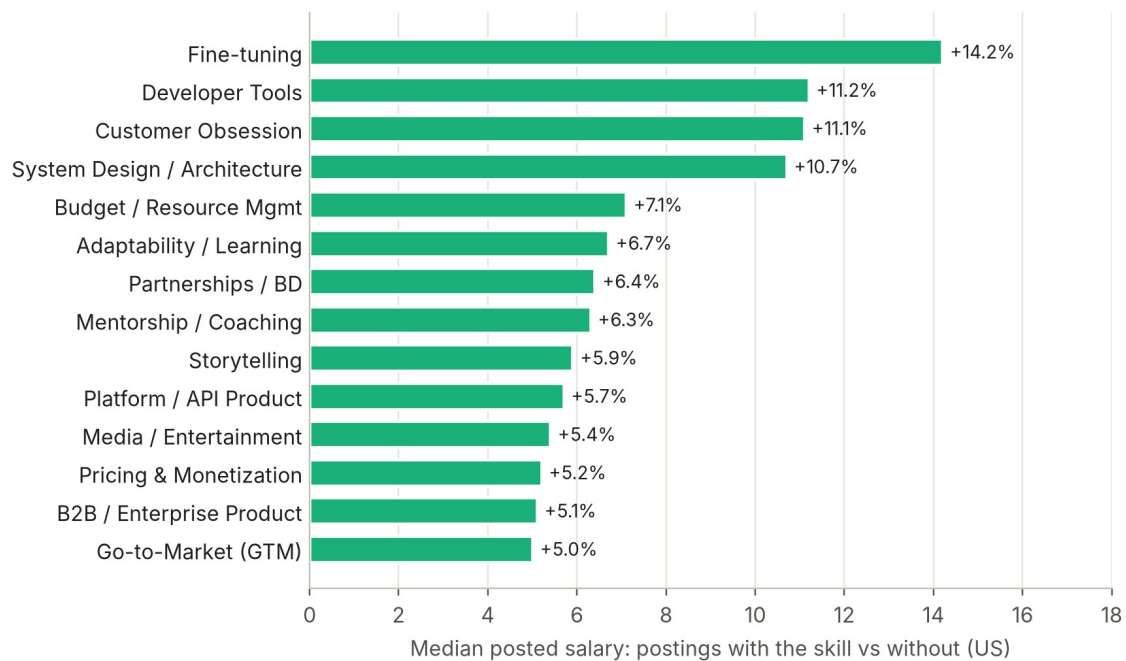


Source: AI Product Manager Hiring Report 2026 (A. Ashtekar), 2,448 unique AI PM postings, CC BY 4.0

Figure 22. India vs US postings: selected skills

Skills that come with higher pay (US)

Correlation, not causation: skills with 40+ US postings that disclose pay



Source: AI Product Manager Hiring Report 2026 (A. Ashtekar), 2,448 unique AI PM postings, CC BY 4.0

Figure 23. Skills associated with higher posted pay in the US (correlation, not causation)

In the \$200K+ segment, the skills that show up more often than in lower-paid roles are **go-to-market (+11.5 pts)**, **mentoring (+10)**, **REST APIs (+9)**, **partnerships/BD (+8)** and **fine-tuning (+7)**. Senior AI PM roles are commercial as much as technical, which suits a consulting background well.

B3. Digest of 44 individual job descriptions

Signature asks are condensed from each posting. Competency codes (C1–C10) map to the competency model in B5 and to the weeks in Part C where you learn them. **Bold** marks asks that recur across many JDs.

#	Company · role · location	What stands out	Skills
1	Anthropic · PM, Claude Code Model Performance · San Francisco	- Drive model launches end-to-end; define readiness criteria - Design agentic evals measuring real-world coding performance; read user transcripts - Must have personally built agentic evals (SWE-bench-style); daily Claude Code user; engineering background + 2+ yrs PM	C1 C4 C5 C8
2	Anthropic · PM, Consumer / Core Product (Claude.ai) · SF / NYC	- Own how new model capabilities show up in the product and the core interaction model 6+ yrs PM, mostly consumer, products with millions of users; craft and quality	C7 C8 C10
3	OpenAI · PM, Core Models · San Francisco	- Model planning & launches; build data flywheels, evals and measurement systems - Pair offline evals with online experiments; 'distinguish a useful metric from a convenient one' - Translate user goals into model requirements across retrieval, ranking, tools, training, inference	C1 C5 C9

#	Company · role · location	What stands out	Skills
4	OpenAI · PM, API Agents · San Francisco	5+ yrs; track record building for developers; intuition for clean APIs and primitives - Roadmap for agentic infrastructure; partner with research at a technical level	C2 C3 C7
5	Google DeepMind · Sr PM, Gemini Post-Training	'In the room during training': read evals, analyse outputs, make quality calls with researchers - Turn capabilities into measurable eval goals per model release; 8 yrs	C1 C5 C8
6	Google DeepMind · PM, Gemini API Models	- Developer's advocate in model release; own model lifecycle on Gemini API & AI Studio - Real-time understanding of model performance on real developer workloads; 5 yrs	C2 C5 C8
7	Google DeepMind · PM, Gemini App	5 yrs PM + 2 yrs launching technical products; cross-functional with eng, UX, legal, marketing - US base \$218–237K + bonus + equity	C7 C8 C10
8	Microsoft AI · PM, Copilot	7+ yrs; hands-on delivery of gen-AI features - LLM platform strategy; prototype approaches to improve response quality, factuality, safety - Define measurable evaluations with datasets; run production experiments on tool use	C2 C4 C5 C6
9	Microsoft IDC · Sr PM, Business & Industry Copilots · Hyderabad / Bengaluru	- Copilot Studio, Dynamics 365, Power Platform AI; 3–5 yrs PM or engineering - Strategise and build the next wave of AI solutions for multi-billion-dollar businesses	C7 C8 C10
10	Amazon / AWS · Sr PM-Technical, Bedrock & Agentic AI	5+ yrs technical PM, 3+ yrs technical experience; own production services, revenue & adoption (P&L) - Plus: responsible AI, AI security, open-source foundation models	C1 C6 C9 C10
11	Meta · PM (AI-native roles)	'Comprehensive evals and data strategies' for continuous improvement - Responsible-AI practice (risk assessment, bias mitigation); prompt/context engineering; agent orchestration - Interview loop now includes an AI product sense round with live vibe-coding of a prototype	C4 C5 C6
12	Glean · PM, AI Quality / Model Hub	- Evaluate LLMs; roadmap for the LLM portfolio; manage inference/model providers Own projections of LLM usage, cost and capacity; eval pipelines, LLM judges, agent observability	C1 C5 C9
13	Perplexity · PM, Growth / SMB	4–10+ yrs; hands-on AI experience (built LLM features) - Build skills, evals and agent systems that make workflows reliable and performant	C3 C5 C9
14	Sierra · PM, Agent Development	5–7 yrs PM; build & ship enterprise agents handling thousands of conversations a day - Explain technical AI concepts to technical and non-technical audiences	C3 C10
15	Decagon · Senior Agent PM	6+ yrs as PM, consultant, engagement manager or founder: consulting backgrounds explicitly welcome - Deep technical acumen to shape agent designs; embed with strategic customers; MBA a plus	C3 C7 C10

#	Company · role · location	What stands out	Skills
16	Intercom (Fin) · Senior / Staff AI PM	- Vision for the core AI systems behind Fin; work with ML scientists No minimum years: 'impact, drive and potential'; strong AI and technical fluency	C1 C2 C7
17	Figma · PM, AI Platform	5+ yrs; proven record shipping and iterating AI features - Nice to have: model training, evaluation and iteration experience	C5 C7
18	Notion · PM, AI Products	Own vision and strategy for AI functionality used by millions; lead cross-functional teams	C7 C10
19	Adobe · Principal PM, Firefly	8–10 yrs PM; gen-AI product experience; some roles need 2+ yrs hands-on with fine-tuning or inference pipelines	C1 C7 C9
20	Salesforce · PM, Agentforce (Commerce, Supply Chain)	- Backlog for semantic retrieval & agentic workflows; PRDs, user stories, acceptance criteria for AI teams - Enterprise SaaS: permissions, integrations, data governance	C3 C8
21	Atlassian · Principal PM, Rovo Search & AI Quality	Vision and roadmap for AI quality in enterprise search and search agents; reliability and trust	C2 C5
22	Uber · PM II Conversational AI Platform / Sr PM AI Quality · Bengaluru	3+ yrs PM (1+ AI) / 5+ yrs (2+ AI) Evaluation infrastructure, benchmarking and observability for agentic support AI	C3 C5 C9
23	Spotify · PM, AI Observability / Evaluation Platform	Platform tools to track LLM behaviour in production, resolve issues fast, measure and refine AI products	C5 C9
24	ASAPP · Sr PM, Observability Tools	Built products around LLMs/agents; shipped tools that simulate, test or preview system behaviour before production	C3 C5
25	HighLevel · Group PM, AI	Adoption, reliability, quality and monetisation of voice/chat agents (calls, booking, lead qualification)	C3 C9
26	Mindtickle · Group PM	Ship fast with small teams; run an eval discipline for an AI product in production	C5 C8
27	TTEC Digital · PM, Services	'Own eval and guardrails as product': define what 'good' and 'safe' mean for each AI service, before and after ship	C5 C6
28	Razorpay · PM II, AI · Bengaluru	- Find high-impact AI problems across the company; prototype and scale with DS/Eng; PRDs, sprint planning 'Claude Code or Cursor as your default environment, orchestrate agents, build with skills and MCPs'	C3 C4 C7
29	PhonePe · AI Product Manager (ML) · Bengaluru	3–7 yrs; LLMs, MCP, RAG, agents, evaluation, NLP, prototyping	C2 C3 C4 C5
30	Paytm · PM, Lending (AI & Platform) / Sr PM, AI Agents	- Re-imagine underwriting, risk, ops and journeys with LLMs and agents 'Something real built that you can explain'; SQL/Excel; agent frameworks (LangGraph, AutoGen); voice & telephony	C3 C4 C9
31	Swiggy · PM I, AI (driver experience) · Bengaluru	2–4 yrs; ship AI-led capabilities incl. Voice AI; LLMs in support flows to cut resolution time	C2 C7
32	Sarvam AI · PM (Models / APIs / Edge) · Bengaluru	'Builders first': talk to users, write specs, ship weekly, instrument funnels - Prompt design, model selection, latency/failure modes, eval pipelines, occasionally write code	C1 C2 C4 C5

#	Company · role · location	What stands out	Skills
33	Freshworks · Sr / Staff PM, Freddy AI · Chennai / Bengaluru	2+ yrs managing AI/ML/NLP products; experimentation to production - AI agents & copilots in CX/ITSM; MBA preferred (Staff)	C3 C7 C9
34	Mastercard · Lead Agentic AI PM / Manager PM, Agentic Commerce	- Requirements for LLMs, RAG, orchestration, tool use; embeddings, vector search, eval frameworks; A/B testing - Payments domain (tokenisation, wallets); firsthand use of agentic coding tools with a portfolio; CS + MBA a plus	C2 C3 C4 C9
35	Visa · Sr PM, Agentic Commerce	- Agent-driven shopping via tokenisation, programmable controls and agentic APIs 8+ yrs (or 5+ with advanced degree)	C3 C7 C10
36	JPMorgan Chase · AI/ML PM (VP) / AI PM, AI Agents	LLMs, agents, RAG pipelines; chain-of-thought prompting strategies; Applied AI CoE platforms	C1 C2 C3
37	Capital One · PM, Enterprise AI/ML	ML platforms, MLOps, cloud (AWS), Kubernetes familiarity; Agile backlog management	C1 C8
38	American Express · Sr PM, ML & Personalization	Hands-on experience developing ML models; applying AI to digital products; Agile	C1 C8 C9
39	Stripe · Staff PM, ML Foundations & GenAI / AI PM, Professional Services	7+ yrs ML-powered products; incubate AI applications; agentic capabilities across the product suite - Technical enough to hold your own in engineering conversations	C1 C3 C7
40	Accenture · AI Product Management Practitioner · India	5 yrs PM, 1 yr platform products, 1+ yr Python/ML frameworks; prompt engineering is a must-have	C1 C2 C8
41	BCG X · AI Product Manager	2–6 yrs; build digital & AI products for clients; analytical and communication strength	C7 C10
42	McKinsey QuantumBlack · Senior Product Analyst	2+ yrs building digital & AI products; lead agile teams; advise senior clients	C7 C10
43	Walmart · Sr PM, Gen-AI (Walmart Connect) / Global Tech India	Own vision and execution of GenAI products; GenAI/agentic experience strongly preferred	C7 C9
44	Flipkart · PM, ML & GenAI Platform org · Bengaluru	4–9 yrs PM; adjacent GenAI roles ask for RAG, LLMs, multimodal applications	C1 C2

B4. Ten patterns across the JDs

- Evals are the new PRD.** Anthropic, OpenAI, Google DeepMind, Microsoft, Glean, Uber, Spotify, Mindtickler and TTEC all put evaluation at the centre of the role. 'Define what good and safe mean... and how it's measured before and after ship' (TTEC).
- The 'builder PM' is now expected.** Razorpay wants Claude Code or Cursor as your default environment; Mastercard wants 'firsthand experience using agentic coding tools... with personal or professional projects'; Paytm wants 'something real built that you can explain'; Sarvam wants PMs who ship weekly and occasionally write code; Meta's loop includes live vibe-coding.
- Agents are the largest category.** 30% of postings are agentic AI roles; 51% mention agents. Tool use, orchestration, human-in-the-loop and agent reliability show up across Sierra, Decagon, Salesforce, Paytm, Visa and Mastercard.
- Model launches are a PM job.** At labs and API platforms, PMs own readiness criteria and model launches, and translate new capabilities into product (Anthropic, OpenAI, DeepMind).

5. **Read the transcripts.** 'Talk with users and analyze transcripts to understand capability gaps' (Anthropic); 'reading evaluations, analyzing model outputs' (DeepMind). Error analysis is a daily habit, not a quarterly review.
6. **Own the quality × latency × cost triangle.** OpenAI lists 'model quality, usefulness, latency, safety, reliability, and cost'; Glean wants LLM usage, cost and capacity projections; AWS wants P&L ownership.
7. **Domain + AI beats generic AI.** Financial services is mentioned in 40% of postings and is the #2 hiring industry. Payments JDs (Mastercard, Visa, Paytm, Razorpay, PhonePe) combine agents and RAG with tokenisation, lending, disputes and compliance.
8. **Responsible AI is a stated requirement.** 23% of postings mention it; Meta, AWS, Accenture and every FS JD ask for risk assessment, bias mitigation and governance.
9. **Classic PM craft still dominates.** Roadmapping (76%), communication (71%), prioritisation (67%), stakeholder management (64%). No JD hires an AI enthusiast who can't run a product.
10. **Consulting and non-linear paths are accepted when paired with AI proof.** Decagon lists 'engagement manager, consultant, founder'; Intercom sets no minimum years; BCG X, QuantumBlack and Accenture hire AI PMs directly. The portfolio is the bridge.

B5. The AI PM competency model

Distilled from the JDs above, these are the ten competencies you'll build. For each: what Level 1 (aware), Level 2 (working) and Level 3 (strong: the hiring bar for most mid-level roles) look like, and where the learning path covers it. **Your goal by Week 24: Level 3 in C2, C3, C4, C5 and C7–C10; Level 2+ in C1 and C6.**

Competency	L1 · Aware	L2 · Working	L3 · Strong (hire bar)	Where
C1 · AI & LLM foundations	Can explain ML vs LLM, tokens, context windows, hallucination	Explains training pipeline, benchmarks, model trade-offs to execs; picks models with data	Reads model cards; shapes model requirements with researchers	Wks 1–4
C2 · Building with LLMs (prompts, APIs, RAG)	Writes decent prompts in a chat UI	Writes versioned system prompts; calls APIs; builds a RAG prototype	Designs context strategy; diagnoses retrieval vs generation failures	Wks 5–8
C3 · Agents, tools & MCP	Knows what an agent and a tool call are	Builds a tool-using agent; uses/builds an MCP server; knows the 6 patterns	Chooses autonomy levels, HITL points and agent UX; threat-models agents	Wks 9–12
C4 · Prototyping & builder skills	Uses no-code AI builders	Ships working demos with Claude Code/Cursor; deploys them	Prototypes in hours to settle debates; vibe-codes live in interviews	Wks 6, 8, 10–12, 21
C5 · Evals & AI quality	Knows offline vs online evals	Builds golden set, rubric, LLM judge; runs error analysis	Calibrates judges; owns quality dashboards & ship gates; eval-driven roadmap	Wks 13–16
C6 · Responsible AI, safety & compliance	Can name failure modes	Runs a red-team; writes guardrail spec & launch checklist	Leads risk reviews with legal/model-risk; knows DPDP/RBI/EU AI Act implications	Wks 15–16
C7 · Discovery & AI strategy	Knows JTBD & user interviews	Runs AI-fit checks; writes opportunity memos; build/buy/partner	Articulates moats and platform-shift strategy; multi-quarter AI bets	Wks 4, 17
C8 · Specs, execution & launches	Writes a PRD	Writes AI PRDs with behaviour spec, eval plan, rollout plan	Runs model launches and upgrades; plans under uncertainty	Wks 18, 20

Competency	L1 · Aware	L2 · Working	L3 · Strong (hire bar)	Where
C9 · Metrics, data & economics	Defines basic KPIs	SQL, A/B design, metrics tree; computes cost per task	Owens unit economics, pricing and margin; experiment programme	Wks 6, 14, 19
C10 · Leadership, communication & GTM	Clear writing	Stakeholder updates; storytelling with eval data; GTM plan	Influences without authority across research, risk and sales; secures exec sponsorship	Wks 19–24

B6. What interviews actually test

JDs describe the job; interview loops test a slightly different mix. Based on 2026 interview guides (Aced/Exponent, IGotAnOffer, KORE1, Northeastern University career centre, and the open-source ai-pm-interview-prep question bank):

Round	Approx. weight	What they test	Prepared in
Behavioural / leadership	~35%	Ownership, conflict, influence without authority; AI failure stories (a launch that hallucinated, a metric that misled)	Week 23
AI product sense	~20%	Design an AI feature end-to-end: user, AI-fit, UX for uncertainty, evals, risks. Some loops (e.g., Meta) add live prototyping	Weeks 17, 20, 23
Technical depth (no coding)	~15%	RAG vs fine-tune, tokens & cost, hallucination mitigation, agents, evals, precision/recall	Weeks 1–16, 23
Execution & metrics	~15%	North-star for an AI feature; diagnose 'users say it's confident but wrong'; A/B design; metric drops	Weeks 14, 18, 23
Strategy / presentation / take-home	~15%	Build vs buy, moats, pricing; an AI PRD or case presentation in 48 hours	Weeks 17–19, 23

B7. Your angle: financial services and payments

This repo already holds your India payments & FS job tracker and a pricing & GTM guide for FS, so the plan below uses **payments and financial services as the default domain** for projects. That is a strong wedge, for four reasons:

- **Demand.** Financial Services & FinTech is 17% of AI PM postings (#2 industry) and appears in 40% of JD texts. JPMorgan, Capital One, Citi, BNY, Intuit and Airwallex are top-20 employers.
- **Live frontier.** Agentic commerce (Visa, Mastercard, UPI pilots), AI in lending (Paytm), AI for merchants (Razorpay) and AI-first support (PhonePe) are hiring right now in India.
- **Regulation is a moat for you.** Most AI builders don't understand RBI, NPCI, DPDP or model-risk expectations. An AI PM who does is rare.
- **Consulting transfer.** Pricing, GTM, process redesign and stakeholder management are exactly the skills that separate \$200K+ AI PM roles (GTM +11.5 pts, partnerships +8 pts).

If your interests change: every project in Part C has a domain slot. Swap payments for healthcare, e-commerce, SaaS or ed-tech and the steps stay the same.

Part C · The 24-week learning path

This is the step-by-step plan. It assumes you are starting from a **business / consulting / MBA background with no coding**, can give it **12–15 hours a week**, and want to be interview-ready for mid-level AI PM roles in about six months. If you already code, compress Phases 1–2 into three weeks. If you have less time, stretch each week to ten days rather than skipping steps.

24-week learning path at a glance

About 12–15 hours a week, roughly 320 hours in total; each phase ends with a portfolio artefact

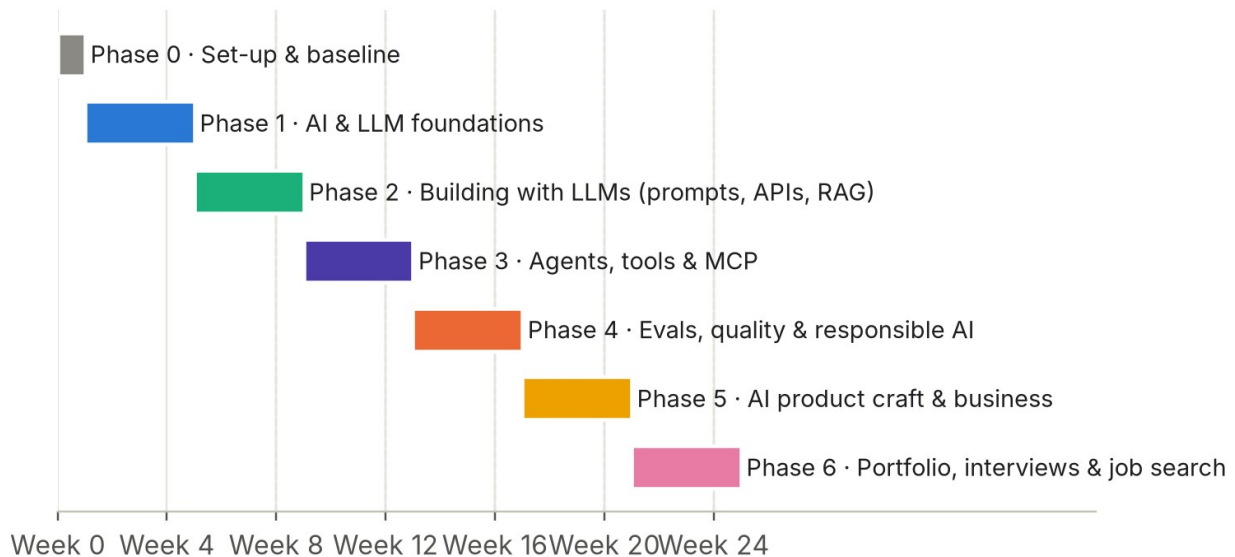


Figure 24. The 24-week plan at a glance

C0. How to run this plan

The weekly rhythm (about 12–15 hours)

Slot	When (suggested)	Length	Purpose
S1	Monday evening	2 h	Learn: read or watch the week's core material, take notes
S2	Wednesday evening	2 h	Learn + try: first hands-on exercise
S3	Friday evening	2 h	Build: main hands-on task
S4	Saturday	3–4 h	Build deep: the week's project work
S5	Sunday	2–3 h	Ship + reflect: finish the deliverable, write the learning-log post, self-check, plan next week

Five rules that make it work

1. **Build every week.** Every week ends in an artefact that goes into your GitHub portfolio. Reading without building doesn't show up in interviews.
2. **Write every week.** A 300–600-word learning-log post (LinkedIn, Substack or Notion) each Sunday. Writing is thinking, and it builds the public proof JDs ask for.
3. **Use AI to learn AI.** Keep Claude/ChatGPT open as a tutor: 'Explain X like I'm a consultant', 'Quiz me on Y', 'Review my PRD as a skeptical staff engineer'. Use Claude Code or Cursor to write code you couldn't write alone. That's exactly how Razorpay, Mastercard and Paytm expect you to work.

4. **Stay in one domain.** Use payments & financial services for every project so your portfolio tells one coherent story.
5. **Re-score yourself monthly** against the competency model (Part B5) and update your JD tracker. Adjust the plan to close the biggest gap.

Budget

You can complete the whole plan for **under ₹10,000** in direct costs: API credits (~\$10–20 across Anthropic/OpenAI/Google, often covered by free tiers), one month of a paid AI assistant or Claude Code/Cursor plan while building projects, and a domain name for your portfolio (optional). All core courses listed are free. Paid courses (e.g., the Maven evals course) are optional accelerators.

Phase 0 · Set-up and baseline (Week 0)

Week 0 · Set up your tools, baseline and tracker

Goal	Have every account and tool ready, know your starting level, and know exactly which roles you're aiming at.
Why (JD evidence)	Every JD assumes daily fluency with AI tools. Your JD tracker turns 'every JD' into your JDs.
Time	6–8 hours

Session	What to do: step by step
S1 1.5 h	• Create accounts: Claude, ChatGPT, Gemini, Perplexity (free tiers are enough to start). • Developer consoles: Anthropic Console, OpenAI Platform, Google AI Studio. Add \$5–10 credit to one of them. • GitHub account; create a public repo ai-pm-portfolio with a README ('What I'm building and learning'). • Google Colab (runs Python in the browser, nothing to install). • Choose your AI coding tool: Claude Code or Cursor. Install it now; you'll use it from Week 6.
S2 1.5 h	• Baseline: score yourself 1–4 on each competency C1–C10 (Part B5). Save as baseline.md in your repo. • Write 'Why AI PM, why me' in 200 words. You'll refine this for interviews. • Pick your target role types (suggest: Agentic AI PM, Applied AI PM in FS, GenAI product PM).
S3 2 h	• Build your JD tracker (Google Sheet). Columns: Company · Role · Link · Location · Years asked · Must-haves · Nice-to-haves · Competency codes · Evidence I have · Gap · Status. • Pull 10 live AI PM JDs from your target list (start from this repo's India payments & FS job tracker, LinkedIn, company career pages). Fill the tracker. • Highlight phrases that repeat. These become resume keywords in Week 22.
S4 1.5 h	• Block your calendar for 24 weeks (S1–S5 slots). • Set up a learning log (LinkedIn posts, Substack or Notion). Write post #0: 'I'm spending 24 weeks becoming an AI PM. Here's the plan.' • Optional: find 1–2 accountability partners (classmates, alumni) and agree a Sunday check-in.

Deliverable: Portfolio repo with README + baseline scores; JD tracker with 10 JDs; calendar blocked; post #0 published.

Self-check (answer out loud, without notes):

- Which three role types are you targeting, and why?
- Which competency is your biggest gap today?

Phase 1 · AI and LLM foundations (Weeks 1–4)

Outcome: you can explain how modern AI works to an executive and to an engineer, choose between models with evidence, and critique a real AI product. Competencies: **C1, C7 (start)**.

Week 1 · Machine-learning fundamentals for PMs

Goal	Understand how machines learn from data, and the metrics that decide whether a model is good for a business.
Why (JD evidence)	'Machine Learning' appears in 52% of postings; Amex, Capital One and JPMorgan ask for ML fluency; interview loops ask precision/recall questions.
Time	12 hours

Session	What to do: step by step
S1 2 h	• Google Machine Learning Crash Course (free, developers.google.com/machine-learning/crash-course/): Introduction, Linear regression, Logistic regression, Classification modules. Skip the code exercises if they slow you down. • Note in your own words: feature, label, training, inference, loss, overfitting, train/validation/test split.
S2 2 h	• AI for Everyone (Andrew Ng, DeepLearning.AI / Coursera; free to audit): weeks 1–2. Focus on 'what AI can and cannot do' and the workflow of an ML project. • Write a 5-line summary of 'the workflow of a machine-learning project'.
S3 2 h	• Hands-on in Google Sheets: a fraud model scored 1,000 UPI transactions, 20 of them actually fraud. Build confusion matrices at three thresholds (e.g., catches 18/20 with 60 false alarms; 14/20 with 15; 10/20 with 3). • Compute precision, recall and F1 for each. Add a business-cost row: a missed fraud costs ₹8,000, a false alarm costs ₹150 in support and churn. Which threshold minimises total cost?
S4 3 h	• Teachable Machine (teachablemachine.withgoogle.com/): train an image classifier on 3 classes in 20 minutes. Deliberately break it (too few examples, biased examples) and note what happens. • Read about data drift and why models degrade after launch (MLCC 'Production ML systems' section). • Optional: Kaggle Learn 'Intro to Machine Learning' lessons 1–3.
S5 2–3 h	• Write a one-page 'ML for PMs' cheat sheet: types of learning, key metrics, when accuracy lies (class imbalance), the precision/recall trade-off, drift. • Learning-log post #1: 'What a fraud model taught me about precision vs recall'.

Deliverable: Confusion-matrix sheet with cost analysis + ML cheat sheet in your repo + post #1.

Done when: You can explain to a CFO why a 99%-accurate fraud model can be useless.

Self-check (answer out loud, without notes):

- What's the difference between precision and recall? Give a payments example where each matters more.
- Why does accuracy mislead on imbalanced data?
- What is overfitting, and how do you detect it?
- Why do models degrade in production?

Week 2 · Neural networks, transformers and how LLMs work

Goal	Build real intuition for tokens, embeddings, attention, pre-training vs post-training, and why LLMs hallucinate.
Why (JD evidence)	'Large Language Models' appears in 46% of postings; technical rounds ask 'explain how an LLM works' and 'why does it hallucinate?'
Time	12–13 hours

Session	What to do: step by step
S1 2 h	• 3Blue1Brown, Neural Networks series (YouTube): Chapter 1 'But what is a neural network?' and Chapter 2 'Gradient descent'. • Then Chapter 5 'But what is a GPT? Visual intro to Transformers'.
S2 2 h	• 3Blue1Brown Chapter 6 'Attention in transformers, visually explained' and Chapter 7 'How might LLMs store facts'. • Draw the flow text → tokens → embeddings → attention layers → next-token probabilities by hand (compare with Figure 3).
S3 2 h	• Andrej Karpathy, '[1hr Talk] Intro to Large Language Models' (YouTube). Take notes on: pre-training, fine-tuning, RLHF, scaling laws, tool use, jailbreaks, prompt injection.
S4 3–4 h	• Tokenizer lab: paste the same sentence in English, Hindi and Hinglish into a tokenizer tool (OpenAI's tokenizer page or tiktokenizer). Record token counts in a table and compute the cost multiplier. • Embeddings: read Jay Alammar's 'The Illustrated Word2vec' and 'The Illustrated Transformer' (jalamar.github.io). • Start Karpathy, 'Deep Dive into LLMs like ChatGPT' (about 3.5 hours): watch the first ~90 minutes (pre-training, tokenisation, base models).
S5 2 h	• Write 'How ChatGPT works, explained to my parents' (500 words) with your own diagram. • Learning-log post #2.

Deliverable: Token-cost comparison table (English vs Hindi vs Hinglish) + explainer post with diagram.

Self-check (answer out loud, without notes):

- What is a token? Why do Indic languages cost more?
- What is an embedding, and what is it used for in products?
- Difference between a base model and an instruct/chat model?
- Why do LLMs hallucinate, in one sentence?
- What does temperature control?

Week 3 · The model landscape: choosing models with evidence

Goal	Know the major model families, how they differ on quality, cost, speed and deployment, and run your own model bake-off.
Why (JD evidence)	Glean wants PMs to 'evaluate LLM models' and manage providers; OpenAI and DeepMind PMs own model launches; Sarvam asks for 'model selection'.
Time	12–13 hours

Session	What to do: step by step
S1 2 h	- Finish Karpathy's 'Deep Dive into LLMs': post-training, RLHF, reasoning models, 'Swiss-cheese' capabilities. - Optional: Karpathy's 'How I use LLMs' for practical usage patterns.
S2 2 h	- Build a model landscape sheet covering closed frontier (Anthropic Claude, OpenAI GPT, Google Gemini), open-weight (Meta Llama, Qwen, Mistral, DeepSeek, Gemma), India-focused (Sarovam, Krutrim) and specialist (embeddings, rerankers, speech-to-text, image). - Columns: price per 1M input / output tokens, context window, modalities, speed, open vs closed, hosting options (incl. India data residency via cloud regions). Use Artificial Analysis (artificialanalysis.ai) and provider pricing pages.
S3 2 h	- Benchmarks: learn what MMLU, GPQA, SWE-bench, HumanEval and LMarena measure and don't. Skim one model's system card (Anthropic or OpenAI publish them) and note the safety-eval sections. - Write 5 bullets on 'why benchmarks don't predict performance on my product'.
S4 3–4 h	Bake-off: write 10 realistic prompts from one FS use case (e.g., explain a credit-card statement line, classify a merchant complaint, summarise an RBI circular, draft a dispute response, answer a UPI FAQ in Hindi). - Run each on 3 models (use free chat UIs or AI Studio / Console). Score every answer 1–5 on correctness, completeness, tone; note latency and estimate cost. - Summarise: which model wins on which task, and at what cost?
S5 2 h	- Write a 1-page model selection memo for 'a UPI support assistant at an Indian bank': recommendation, fallback model, data-residency considerations, cost estimate, risks. - Post #3: 'I tested 3 models on 10 banking tasks. Here's what surprised me.'

Deliverable: Model landscape sheet + bake-off scoring sheet + model selection memo.

Self-check (answer out loud, without notes):

- When would you pick an open-weight model over a frontier API?
- What does SWE-bench measure, and why might it not matter for your product?
- What would make you switch models after launch, and what must you re-run first?

Week 4 · How AI products are built, plus your first teardown

Goal	Connect everything into the end-to-end product picture (Part A) and apply it to real products.
Why (JD evidence)	Every product-sense round starts with 'walk me through how you'd build X'. Teardowns are the fastest early portfolio artefact.
Time	12–14 hours

Session	What to do: step by step
S1 2 h	Re-read Part A of this guide. Redraw Figure 1 (the stack) from memory and fill the 'what the PM decides' table for one real product (e.g., Swiggy's support bot, Perplexity, or a bank's chatbot).
S2 2 h	Google PAIR 'People + AI Guidebook' (pair.withgoogle.com/guidebook): chapters on User Needs + Defining Success, Explainability + Trust, Errors + Graceful Failure. - Skim Microsoft's Guidelines for Human-AI Interaction (18 guidelines).
S3 2 h	- Listen: Lenny's Podcast with Kevin Weil (OpenAI CPO) and Mike Krieger (Anthropic CPO). Note three ways they say the PM role is changing. - Read Anthropic's blog post 'Product management on the AI exponential' (claude.com/blog).
S4 3–4 h	Teardown #1 of an AI product you use. Structure: user & job-to-be-done · AI pattern (chat, copilot, agent, embedded) · likely architecture (draw it) · 15 adversarial tests and what broke · guardrails you noticed · feedback loops · pricing · the eval metrics you'd track · 3 improvements. Teardown #2: an Indian FS/payments AI feature (e.g., a bank assistant, a UPI app's support bot, a broker's AI features).
S5 2 h	Phase 1 gate (see box below). Publish both teardowns.

Deliverable: Two published AI product teardowns (Portfolio artefact #1).

Self-check (answer out loud, without notes):

- Walk through the life of a request in an AI assistant (Figure 4) in 2 minutes.
- Name 3 design principles for AI UX from PAIR.
- What's the difference between a copilot and an agent?

Phase 1 gate: are you ready to move on?

- Can you explain tokens, context windows, embeddings, hallucination, pre- vs post-training without notes? (Ask Claude to quiz you with 20 questions; aim for 16+.)
- Portfolio: ML cheat sheet, token-cost table, model bake-off + memo, two teardowns.
- Re-score C1 and C7. Target: C1 = Level 2.

Phase 2 · Building with LLMs: prompts, APIs, RAG, prototypes (Weeks 5–8)

Outcome: you can build and deploy a working AI prototype that answers questions from real documents with citations, and measure how well it works. Competencies: **C2, C4, C9 (start)**.

Week 5 · Prompt and context engineering

Goal	Write production-grade prompts: versioned, tested, with structured output.
Why (JD evidence)	Prompt engineering is a must-have at Accenture and JPMorgan (chain-of-thought); Sarvam lists 'prompt design'; Meta lists 'prompt/context engineering'.
Time	12–14 hours

Session	What to do: step by step
S1 2 h	• Anthropic's interactive prompt-engineering tutorial (GitHub: anthropics/prompt-eng-interactive-tutorial, or the 'courses' repo): chapters 1–4 (structure, clarity, roles, separating data from instructions).
S2 2 h	• Tutorial chapters 5–9: formatting output, thinking step by step, few-shot examples, avoiding hallucinations, complex prompts. • Skim the OpenAI and Google Gemini prompting guides; list 5 techniques all three agree on.
S3 2 h	• Read Anthropic's engineering post 'Effective context engineering for AI agents'. Summarise: what goes into context, what to leave out, and why.
S4 3–4 h	• Write a system prompt v1 for a 'UPI dispute assistant': role, scope (what it must refuse), tone, escalation rules, output format, 5 few-shot examples. • Create a test sheet of 30 inputs (20 normal, 5 edge, 5 adversarial). Run v1 in the Anthropic Console / OpenAI Playground; mark pass/fail. • Iterate to v2 and v3; keep a changelog ('v2: added rule X because cases 7, 12 failed'). Record the pass rate per version.
S5 2 h	• Structured output: make the model return JSON <code>{category, urgency, needs_human, reason}</code> for 50 complaints you've labelled yourself. Measure accuracy vs your labels. • Post #5: 'Prompt v1 → v3: from 60% to 90% pass rate'.

Deliverable: Versioned system prompt (v1–v3) with changelog + 30-case test sheet + JSON classification accuracy.

Self-check (answer out loud, without notes):

- Name 6 prompt-engineering techniques and when each helps.
- Why version prompts and keep a test set?
- What is context engineering, and how does it differ from prompt engineering?

Week 6 · Just-enough Python, APIs and SQL

Goal	Call an LLM API from code, process a dataset, and query data with SQL, using an AI coding assistant to help.
Why (JD evidence)	APIs appear in 28% of postings (the #1 technical ask); SQL in 11%; Paytm asks for SQL/Excel; Razorpay and Mastercard expect AI-assisted building.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	• Python basics: Kaggle Learn 'Python' lessons 1–4 or Python for Everybody (py4e.com) chapters 1–6: variables, lists, dictionaries, loops, functions. • Don't aim for mastery. Aim to <i>*read*</i> code an AI assistant writes for you.

Session	What to do: step by step
S2 2 h	- APIs: what REST, JSON, HTTP methods, status codes, API keys and rate limits are. Call a public API from your browser or Postman. - Read the Anthropic or OpenAI API quickstart. Understand: messages, system prompt, model name, max tokens, temperature, usage (token counts) in the response.
S3 2–3 h	- In Google Colab, with Claude/ChatGPT/Claude Code helping you: call an LLM API from Python, send one prompt, print the answer and the token usage. - Store your API key safely (Colab secrets). Never commit keys to GitHub.
S4 3–4 h	Mini-project, 'batch classifier': load 200 customer complaints from a CSV (create synthetic ones with an LLM if needed), classify each via the API using your Week 5 prompt, write results to a new CSV. - Compute accuracy vs your labels, a confusion matrix, and cost per 1,000 complaints from token usage.
S5 2–3 h	- SQL: SQLBolt lessons 1–12 (or Mode's SQL tutorial basics). Then write 5 queries on a transactions table: failures by bank, daily volume, top merchants by disputes, a JOIN with a merchants table, a 7-day moving average. - Push the notebook + README to GitHub. Post #6.

Deliverable: Colab notebook (batch classifier with accuracy + cost) and 5 SQL queries on GitHub.

Self-check (answer out loud, without notes):

- What is in an LLM API request and response?
- How do you compute cost per task from the usage field?
- What is a rate limit, and what should the product do when it's hit?
- Write a GROUP BY query from memory.

Week 7 · Retrieval-Augmented Generation (RAG), hands-on

Goal	Build a RAG system over real regulatory documents and measure retrieval and answer quality.
Why (JD evidence)	RAG appears in 19% of postings and in Mastercard, JPMorgan, PhonePe and Salesforce JDs. It's the default enterprise AI architecture.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	Re-read Part A5. Then Anthropic's 'Introducing Contextual Retrieval' post and one DeepLearning.AI short course: 'LangChain: Chat with Your Data' or 'Building and Evaluating Advanced RAG Applications'.
S2 2 h	- Collect a corpus: 20–40 public documents, e.g., RBI Master Directions on digital payments and KYC, NPCI UPI dispute and FAQ documents, a bank's public fee schedule. No-code RAG first: load them into NotebookLM. Ask 20 questions; record wrong answers, missing tables, bad citations.
S3 2–3 h	Code RAG in Colab with Claude Code / Cursor helping: parse PDFs → chunk → embed → store in a vector DB (Chroma or similar) → retrieve top-k → answer with citations. LlamaIndex or LangChain can shortcut this. - No-code alternative if stuck: Flowise, Langflow or n8n.

Session	What to do: step by step
S4 3–4 h	• Evaluate: write 40 question-answer pairs with the source section for each (your golden set). • Measure retrieval hit rate@5 (is the right chunk in the top 5?) and answer correctness (your judgement, pass/fail). • Experiment: 2 chunk sizes × with/without reranking or hybrid keyword search. Record a results table.
S5 2 h	• Write a 2-page RAG design doc: sources & freshness, permissions, chunking choice and why, metrics, failure modes (Part A5 table), cost and latency per query.

Deliverable: Working RAG notebook + 40-question golden set + experiment results + RAG design doc.

Self-check (answer out loud, without notes):

- Walk through RAG ingestion and query steps.
- Your RAG bot gives a wrong answer. How do you tell whether retrieval or generation failed?
- When is RAG the wrong solution?

Week 8 · Prototyping ('vibe coding') and shipping a demo

Goal	Turn your RAG system into a deployed web app with good AI UX, and test it with real people.
Why (JD evidence)	'Builder PM' is the #2 JD pattern. Meta's loop includes live prototyping; Sarvam wants PMs who ship weekly; Mastercard wants a portfolio.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	• Tour the tools: Lovable, Bolt, v0, Replit (app builders) and Google AI Studio's app builder; and Claude Code / Cursor (AI coding agents). Build a throwaway 'hello world' app in one of each type. • Learn the loop: describe → generate → run → read the diff → correct → commit.
S2 3 h	• Build a front-end for your RAG bot: Streamlit (Python, via Claude Code) or Lovable calling your backend. • Deploy free: Streamlit Community Cloud, Hugging Face Spaces or Vercel.
S3 2–3 h	• Add AI UX: citations with links, an 'I'm not sure, here's who to contact' state, a 👍/👎 control logging to a Google Sheet, a 'talk to a human' button, a disclaimer where needed.
S4 3 h	• Usability test with 5 people (classmates, friends in banking). Give them 3 tasks; watch silently; log 20 observations. • Fix the top 3 issues.
S5 2–3 h	• Phase 2 gate. Publish Portfolio project #2: 'Payments Policy Copilot'. README: problem, users, architecture diagram, eval results (from Week 7), cost per query, UX decisions, what you'd do next. Record a 2-minute Loom demo.

Deliverable: Deployed RAG app with public URL + README + demo video (Portfolio project #2).

Self-check (answer out loud, without notes):

- Demo your app and explain one design trade-off in 3 minutes.
- What did usability testing reveal that evals missed?

Phase 2 gate

- Live app URL that a recruiter can click. README with numbers (hit rate, accuracy, cost per query).
- You can build a small app with an AI coding tool without a tutorial.
- Re-score C2, C4. Target: both at Level 2.

Phase 3 · Agents, tools and MCP (Weeks 9–12)

Outcome: you have designed, built, tested and hardened a tool-using agent for a real payments workflow, with human-in-the-loop controls. Competencies: **C3, C4, C6 (start)**.

Week 9 · Agent fundamentals and patterns

Goal	Understand tool calling, the agent loop, the six patterns, and autonomy levels, and pick your agent use case.
Why (JD evidence)	30% of AI PM roles are agentic; 51% of postings mention agents; Sierra, Decagon, Paytm, Visa and Mastercard hire specifically for agents.
Time	12–14 hours

Session	What to do: step by step
S1 2 h	• Read Anthropic's 'Building effective agents'. Summarise each of the 6 patterns (Figure 8) with a payments example. • Read OpenAI's 'A practical guide to building agents' (PDF). Note their guidance on guardrails and human intervention.
S2 2–3 h	• Tool-use docs (Anthropic 'tool use' or OpenAI 'function calling'). In Colab, build a 2-tool agent: <code>get_fx_rate(currency)</code> (mock) and <code>calc_remittance_fee(amount, corridor)</code>. Watch the loop in the logs.
S3 2 h	• DeepLearning.AI: 'Agentic AI' (Andrew Ng), modules on reflection, tool use, planning and multi-agent; or 'AI Agents in LangGraph'.
S4 3 h	• List 10 agent use cases in payments/FS (dispute resolution, merchant onboarding/KYC, reconciliation, collections outreach, chargeback evidence, invoice-to-pay, fraud-alert triage, loan document checks, treasury reporting, customer refunds). • Score each on value, frequency, error cost, data access and autonomy level (L1–L5). Pick one for Project #3. The default suggestion is the dispute/chargeback triage agent.
S5 2 h	• Write 'The autonomy spectrum for payments agents' (post #9) using your scoring table.

Deliverable: 2-tool agent notebook + scored use-case table + chosen agent project.

Self-check (answer out loud, without notes):

- What is the difference between a workflow and an agent?
- How does tool calling actually work (who executes the tool)?
- Which autonomy level fits a refund agent, and why?

Week 10 · MCP, integrations and agent security

Goal	Use and build MCP servers, automate a workflow, and threat-model your agent.
Why (JD evidence)	MCP is named in 9% of postings and explicitly by Razorpay and PhonePe; OWASP's LLM risks show up in every security review.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	modelcontextprotocol.io: introduction and core concepts (hosts, clients, servers; tools, resources, prompts). - DeepLearning.AI short course 'MCP: Build Rich-Context AI Apps with Anthropic', or Anthropic Academy's MCP course.
S2 2 h	Use MCP as a user: connect Claude Desktop (or another MCP host) to 2 servers (e.g., filesystem, Google Drive or GitHub). Complete a real task through them.
S3 3 h	Build a tiny MCP server in Python (e.g., with the official SDK's FastMCP helper) exposing <code>get_transaction(txn_id)</code> and <code>list_disputes(merchant_id)</code> over a CSV. Let Claude Code write it; you review and run it. Connect it to your MCP host and query it in plain English.
S4 3 h	- Workflow automation: in n8n or Zapier, build 'email arrives → LLM classifies → create ticket → draft reply → human approves → send'. - Note which steps are deterministic and which are model-driven.
S5 2–3 h	- Security: read the OWASP Top 10 for LLM Applications and Simon Willison's writing on prompt injection and the 'lethal trifecta' (private data + untrusted content + ability to communicate externally). - Write a threat model for your agent: assets, attackers, injection paths, excessive-agency risks, mitigations.

Deliverable: Working MCP server in repo + n8n/Zapier workflow screenshot + agent threat model.

Self-check (answer out loud, without notes):

- Explain MCP to a bank CTO in 60 seconds.
- What is indirect prompt injection? Give a payments example.
- What is the 'lethal trifecta', and how do you break it?

Week 11 · Build the agent (Portfolio project #3)

Goal	Spec and build a working dispute-resolution agent with tools, RAG, human approval and tracing.
Why (JD evidence)	'Something real built that you can explain' (Paytm); 'build and ship AI agents' (Sierra); 'orchestrate agents, build with skills and MCPs' (Razorpay).
Time	14–16 hours

Session	What to do: step by step
S1 2 h	Write a short agent spec (use the AI PRD template in Appendix C): goal, users, tools, autonomy level, HITL points, guardrails, success metrics, cost budget.

Session	What to do: step by step
S2 3 h	Build the core loop with Claude Code (or OpenAI Agents SDK / LangGraph / n8n). Tools: <code>lookup_txn</code>, <code>search_policy</code> (your Week 7 RAG), <code>classify_reason_code</code>, <code>draft_evidence</code>, <code>request_human_approval</code>, <code>send_email</code> (mock).
S3 3 h	Add tracing with a free tier of Langfuse, LangSmith or Arize Phoenix, so every step, tool call, token count and latency is visible.
S4 4 h	- Write 30 test scenarios: 18 normal disputes, 6 edge cases (missing data, duplicate claims), 6 adversarial (prompt injection hidden in a merchant email, requests to exceed refund limits). - Run all 30; save traces.
S5 2 h	Measure: task success rate, average steps per task, cost per task, % escalated to human, unsafe-action attempts blocked. Put these in a table in the README.

Deliverable: Working agent in repo, traces, 30-scenario results table.

Self-check (answer out loud, without notes):

- Walk through one trace step by step.
- Where is the human in the loop, and why there?
- What's your agent's cost per resolved dispute?

Week 12 · Agent UX, reliability and hardening

Goal	Make the agent trustworthy: good UX for autonomy, reliability engineering, and fixes for its top failure modes.
Why (JD evidence)	Agent JDs stress reliability and adoption (HighLevel: 'adoption, reliability, quality'). UX for autonomy is a common product-sense question.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	Agent UX patterns: show the plan before acting, approvals, undo, progress, explanations, audit trail. Study how Claude Code, Cursor or an agentic browser handle permissions. Read PAIR's 'Feedback + Control' chapter.
S2 2 h	Reliability: timeouts, retries, max-step limits, idempotent tools (no double refunds!), fallbacks when a tool fails, spend caps.
S3 3–4 h	Error analysis on your 30 traces: label each failure, group into types, count. Fix the top 3 (prompt, tool design or guardrail). Re-run all 30 and build a before/after table.
S4 3 h	- Add a simple UI (Streamlit) showing plan → approval → result, with an audit log view. - Record a 3-minute demo.
S5 2–3 h	Phase 3 gate. Publish Portfolio project #3: 'Dispute Resolution Agent' with architecture diagram, threat model, results table and demo. - Practise out loud: 'Design an AI agent for merchant onboarding at Razorpay' (35 minutes, recorded).

Deliverable: Hardened agent + before/after results + demo (Portfolio project #3).

Self-check (answer out loud, without notes):

- How do you prevent an agent from refunding twice?
- Which 3 UX patterns build trust in an agent?
- What were your agent's top 3 failure modes and fixes?

Phase 3 gate

- Two deployed projects (RAG copilot, agent) with numbers in the README.
- You can explain tool calling, MCP, agent patterns and prompt injection crisply.
- Re-score C3, C4. Target: Level 2–3.

Phase 4 · Evals, quality and responsible AI (Weeks 13–16)

Outcome: you can design and run an eval programme and a red-team, calibrate an LLM judge, and write a responsible-AI launch checklist for a regulated product. This is the most in-demand technical skill in the JDs. Competencies: **C5, C6, C9**.

Week 13 · Eval fundamentals and error analysis

Goal	Learn the eval toolkit and run proper error analysis on your own product.
Why (JD evidence)	Evals are a must-have in 66% of JDs that split requirements; central at Anthropic, OpenAI, DeepMind, Microsoft, Glean, Uber, Mindtickler, TTEC.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	• Read: Hamel Husain, 'Your AI Product Needs Evals'; Lenny's Newsletter, 'Beyond vibe checks: A PM's complete guide to evals' (Aman Khan); Anthropic, 'Demystifying evals for AI agents'.
S2 2 h	• Learn the vocabulary: code-based vs LLM-judge vs human graders; reference-based vs reference-free; pointwise vs pairwise; offline vs online; pass@k; faithfulness, relevance, task success. Make flashcards.
S3 3–4 h	• Error analysis (the Husain/Shankar method) on Project #2 or #3: take 100 traces, write a free-text note on each failure ('open coding'), then group notes into 5–8 failure categories ('axial coding'), count them, and chart a Pareto.
S4 3 h	• Build golden dataset v1: 100 cases (60 typical, 25 edge, 15 adversarial), each with the expected behaviour and the rubric criteria it tests.
S5 2 h	• Write a 1-page quality bar: must-pass criteria (safety, PII: 100%), target criteria (e.g., correct resolution ≥ 90%), ship gate, and who signs off.

Deliverable: Error-analysis Pareto + 100-case golden set + quality-bar doc.

Self-check (answer out loud, without notes):

- Offline vs online evals: what does each catch?
- Why start with error analysis rather than generic metrics?
- What goes in a golden dataset?

Week 14 · LLM-as-judge, eval tooling and experiments

Goal	Build calibrated LLM judges, automate regression evals, and design an A/B test for an AI feature.
Why (JD evidence)	Glean: 'LLM-powered judges'; OpenAI: 'pair offline evaluation with online experimentation'; Mastercard: 'experimentation, A/B testing'; Experimentation is in 20% of postings.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	Write 3 judge prompts: faithfulness, policy compliance, tone. Each returns PASS/FAIL plus a one-line critique. Binary beats 1–10 scales.
S2 3 h	Calibrate: hand-label 50 outputs yourself. Run the judges. Compute agreement, true-positive and true-negative rates. Iterate the judge prompts until agreement $\geq 85\text{--}90\%$. Record each iteration.
S3 3 h	Tooling: set up one eval tool (Promptfoo is free and CLI-based; or Braintrust, LangSmith, Langfuse or Arize Phoenix). Run a comparison: prompt v3 vs v4, and model A vs model B, on your golden set.
S4 3 h	Build a quality dashboard (Sheet or Looker Studio): pass rate by criterion and category, trend by version, cost and latency per version.
S5 2 h	Experimentation: read Kohavi, Tang & Xu, 'Trustworthy Online Controlled Experiments', chapters 1–3 (or summaries). Write an A/B test plan for your copilot: hypothesis, randomisation unit, primary metric (e.g., resolution without human), guardrails (complaints, cost, latency), sample size (online calculator), duration, decision rule.

Deliverable: 3 calibrated judges with agreement stats + automated eval run + dashboard + A/B test plan.

Self-check (answer out loud, without notes):

- How do you know your LLM judge can be trusted?
- Design a north-star and 3 guardrail metrics for an AI support bot.
- What's the randomisation unit for an A/B test of a chatbot, and why?

Week 15 · Safety, guardrails and red-teaming

Goal	Attack your own product, add layered guardrails, and write an incident playbook.
Why (JD evidence)	Guardrails & safety in 13% of postings; responsible AI in 23%; TTEC: 'own eval and guardrails as product'; every FS JD expects risk discipline.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	Review Part A11. DeepLearning.AI short courses 'Red Teaming LLM Applications' and/or 'Quality and Safety for LLM Applications'.
S2 3 h	Red-team Project #2/#3 with 50 attacks across: prompt injection (direct and hidden in documents/emails), jailbreaks, PII extraction, off-topic misuse, excessive agency, and over-refusal checks. Promptfoo has a red-team mode that can generate attacks. Log the success rate by category.
S3 3 h	Implement guardrails: input classifier for injection/off-topic, PII masking (e.g., Microsoft Presidio), output groundedness check, refusal and handoff UX. Re-run the red-team; record before/after.

Session	What to do: step by step
S4 2–3 h	Fairness in FS: for a lending or credit use case, learn disparate impact, protected attributes, proxy variables and explainability. Write when AI should <i>*never*</i> decide alone (tie to the error-cost matrix, Part A9).
S5 2 h	- Write an AI incident playbook: severity levels, detection, kill switch, comms, customer remediation, post-mortem template. - Read one public AI post-mortem (e.g., Anthropic's engineering post-mortems) and the 2024 Air Canada chatbot ruling. Note the lessons for PMs.

Deliverable: Red-team report (50 attacks, before/after) + guardrail implementation + incident playbook.

Self-check (answer out loud, without notes):

- Name 5 LLM failure modes and a defence for each.
- Your bot was tricked into revealing another user's data. Walk through your first 24 hours.
- Why is over-refusal a product problem, not just a safety win?

Week 16 · Responsible AI and regulation, plus eval report

Goal	Know India-first and global AI rules well enough to lead a risk review; publish your eval & red-team work.
Why (JD evidence)	Regulation-aware AI PMs are scarce in FS; 'Legal / RegTech' appears in 34% of postings; AWS, Meta and Accenture list responsible-AI experience.
Time	12–14 hours

Session	What to do: step by step
S1 2–3 h	India: DPDP Act 2023 and DPDP Rules (notified Nov 2025): consent, notice, purpose limitation, rights, significant data fiduciaries. RBI FREE-AI committee report (Aug 2025): principles and recommendations for AI in financial services. MeitY India AI Governance Guidelines (Nov 2025). Summarise each in 10 bullets.
S2 2 h	Global: EU AI Act risk tiers and why credit scoring is high-risk; NIST AI RMF (Govern, Map, Measure, Manage) and its Generative AI Profile; ISO/IEC 42001.
S3 2 h	- Write a Responsible AI launch checklist (1 page) for your copilot/agent: data & consent, model risk, fairness, transparency, human oversight, security, monitoring, grievance redressal. - Write a model/system card for Project #2.
S4 3 h	Publish Portfolio project #4: 'Eval & red-team report' for your copilot/agent: error taxonomy, golden set design, judge calibration, red-team results, guardrails, before/after metrics, launch checklist.
S5 2 h	Phase 4 gate mock: explain your eval approach for a bank chatbot in 10 minutes; then answer 'users say it's confident but wrong. What do you do?'

Deliverable: Eval & red-team report (Portfolio project #4) + RAI checklist + system card.

Self-check (answer out loud, without notes):

- What would the RBI expect from a bank deploying a customer-facing AI assistant?
- How does DPDP affect sending customer chats to a foreign model API?
- Map your product to the EU AI Act risk tiers.

Phase 4 gate

- You can design an eval programme from scratch and defend it under questioning.
- Four portfolio artefacts: teardowns, RAG copilot, agent, eval & red-team report.
- Re-score C5, C6. Target: C5 = Level 3, C6 = Level 2+.

Phase 5 · AI product craft and business (Weeks 17–20)

Outcome: you can take an AI opportunity from discovery to a fundable plan, with PRD, metrics, launch plan, unit economics, pricing and GTM. This is where your consulting background becomes a weapon.

Competencies: **C7, C8, C9, C10**.

Week 17 · Discovery and AI strategy

Goal	Find a real, validated AI opportunity through user research and frame the strategy.
Why (JD evidence)	Product strategy (53%), customer empathy (34%), market research and competitive analysis all recur; Razorpay asks PMs to 'find high-impact problems where AI can create step-change impact'.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	• Teresa Torres, 'Continuous Discovery Habits': opportunity-solution trees and interviewing chapters. • Rob Fitzpatrick, 'The Mom Test' for non-leading questions.
S2–S3 4 h	• Run 5 user interviews on one workflow (e.g., small merchants handling disputes, ops analysts reconciling payments, relationship managers at a bank). Use your institute and alumni network. • Map the workflow step by step, with time spent and pain points. Mark where AI could help using the AI-fit flow (Figure 10).
S4 3 h	• Strategy: build vs buy vs partner; moats (data, distribution, workflow integration, trust/compliance). Read Chip Huyen's 'AI Engineering', chapter 1 (use-case evaluation). Read one investor essay on AI application moats (e.g., Sequoia's 'Generative AI's Act o2'). • Build a competitive landscape table (incumbents, AI-native startups, in-house bank builds).
S5 2–3 h	• Write a 2-page opportunity memo: problem, evidence (quotes, numbers), who pays, why AI, why now, why us, risks, first experiment.

Deliverable: Interview notes + workflow map + opportunity memo + competitive table.

Self-check (answer out loud, without notes):

- What's the opportunity-solution tree for your problem?
- Isn't this just a wrapper on GPT/Claude? Answer in 60 seconds.
- When should a bank buy rather than build an AI assistant?

Week 18 · The AI PRD, metrics and launch plan

Goal	Write a complete AI PRD with behaviour spec, eval plan, metrics tree and staged rollout plan.
Why (JD evidence)	PRDs in 35% of postings (46% in India); Salesforce wants 'PRDs, user stories and acceptance criteria for AI teams'; Anthropic wants 'readiness criteria' for launches.
Time	13–15 hours

Session	What to do: step by step
S1 2 h	Study the AI PRD template (Appendix C). Compare it to a classic PRD. List the 6 sections that are AI-specific.
S2–S3 5 h	- Write the full AI PRD for your capstone. Suggested capstone: 'Agentic dispute & refund assistant for UPI merchants' or 'AI copilot for bank relationship managers'. - Include: problem & evidence; users & JTBD; solution & AI pattern; behaviour spec (must / must-not / escalate); data & context; eval plan & ship gate; metrics; risks & guardrails; cost budget; rollout; open questions.
S4 3 h	Metrics tree: north star (e.g., disputes resolved without human within 24 h) → input metrics → AI-quality metrics (resolution accuracy, groundedness) → guardrails (complaint rate, wrong-refund rate, cost/task, p95 latency) → business metrics. Rollout: internal dog-food → 1% → 10% → 50% → GA, with success criteria and rollback triggers at each gate.
S5 2 h	Get 2 people to review your PRD (one technical, one business). Revise. Post #18: 'What an AI PRD needs that a normal PRD doesn't'.

Deliverable: Complete AI PRD (8–12 pages) with metrics tree and rollout plan.

Self-check (answer out loud, without notes):

- What's in a behaviour spec?
- Define the north-star metric for your capstone and defend it against 2 alternatives.
- What triggers a rollback?

Week 19 · Unit economics, pricing and go-to-market for AI

Goal	Model the cost, price and GTM for your AI product: the commercial layer where \$200K+ roles differ.
Why (JD evidence)	GTM is the #1 skill separating \$200K+ roles (+11.5 pts); AWS wants P&L ownership; Glean wants cost/capacity projections; HighLevel wants monetisation.
Time	12–14 hours

Session	What to do: step by step
S1 2–3 h	Token math: compute cost per task for your capstone under 3 model strategies (frontier only, routing, small fine-tuned model). Scale to 10k, 100k and 1M users. Add caching and batch savings (Part A12).
S2 2 h	Pricing: seat vs usage vs outcome-based vs hybrid (Part A12 callout). Re-read this repo's pricing guide, chapter 01 (pricing fundamentals). Pick a pricing model and justify it.
S3 2 h	GTM for AI: positioning, proof (pilots, published eval results), enterprise buying (security review, data residency, DPDP), change management, adoption metrics. Re-read this repo's GTM guide, chapter 02.

Session	What to do: step by step
S4 3 h	Build a 1-page business case in a spreadsheet: market sizing, adoption ramp, revenue, model and infra costs, gross margin, customer ROI (e.g., ₹ saved per dispute).
S5 2 h	Add pricing, GTM and business case sections to your PRD. Post #19: 'The unit economics of an AI agent'.

Deliverable: Cost model + pricing recommendation + GTM plan + business case (Portfolio project #5 with the PRD).

Self-check (answer out loud, without notes):

- What's your cost per task and gross margin at 100k users?
- Why might outcome-based pricing work for your product, and what's the risk?
- How do you sell an AI product to a bank's risk committee?

Week 20 · Execution and leadership in AI teams

Goal	Learn to run an AI team day-to-day: roadmaps under uncertainty, model upgrades, stakeholder updates, and core PM craft.
Why (JD evidence)	Roadmapping 76%, prioritisation 67%, stakeholder management 64%, cross-functional leadership 57%: the most-requested skills of all.
Time	12–14 hours

Session	What to do: step by step
S1 2 h	Roadmaps under uncertainty: bets, timeboxed spikes, 'eval-driven development', and a model-upgrade playbook (new model ships → run the eval suite → compare cost/latency → staged switch).
S2 2 h	Classic craft refresh: prioritisation (RICE, Kano), now/next/later roadmaps, OKRs. Marty Cagan's 'Inspired' (key chapters or a good summary) and Shreyas Doshi's LNO framework.
S3 2 h	Write a 1-page exec update for your capstone: status, eval dashboard snapshot, risks, asks. Then a 1-page risk-committee memo in plain language.
S4 3–4 h	Case practice: 3 AI product-sense cases, 35 minutes each, recorded. For example, 'Design an AI feature for PhonePe merchants'; 'Improve Swiggy's support with AI'; 'An AI assistant for Zerodha users: should they build it?' Self-review against the structure in Appendix E.
S5 2 h	Phase 5 gate: PRD + business case complete and reviewed. Re-score C7–C10.

Deliverable: Exec update + risk memo + 3 recorded cases with self-review.

Self-check (answer out loud, without notes):

- How do you roadmap a feature when you don't know if the model can do it?
- A new model is 30% cheaper. What do you do before switching?
- How do you say no to a senior stakeholder's AI idea?

Phase 5 gate

- A PRD + business case you'd be comfortable sending to a hiring manager.
- Three recorded product-sense cases with self-critique.
- Re-score C7–C10. Target: Level 2–3.

Phase 6 · Portfolio, interviews and job search (Weeks 21–24)

Outcome: a capstone that ties everything together, a portfolio and resume that mirror the JDs, interview readiness across all rounds, and a live pipeline of applications. Competencies: **all, especially C10**.

Week 21 · Capstone build

Goal	Build the MVP described in your PRD, combining RAG, agent, evals and UX.
Why (JD evidence)	Portfolios substitute for years of experience. Every JD pattern (builder, evals, agents, domain, economics) shows up in one project.
Time	14–16 hours

Session	What to do: step by step
S1–S3 7 h	• Build the capstone MVP with Claude Code / Cursor, reusing Projects #2–#4 (RAG, agent, eval suite, guardrails). • Keep a build log: decisions, trade-offs, what you cut and why.
S4 4 h	• Run the full eval suite + red-team; produce the eval report; compute real cost per task.
S5 3 h	• Record a 4-minute demo: problem → user → live demo → evals → economics → what's next.

Deliverable: Working capstone + eval report + demo video.

Self-check (answer out loud, without notes):

- Explain the capstone's three hardest trade-offs.
- What would you do with 3 engineers and 3 months?

Week 22 · Portfolio, resume and personal brand

Goal	Package your work so a recruiter understands your value in 30 seconds and a hiring manager in 5 minutes.
Why (JD evidence)	Mastercard: 'showcase personal or professional projects'; Paytm: 'something real built that you can explain'.
Time	12–14 hours

Session	What to do: step by step
S1 3 h	• Portfolio site (Notion, Framer, GitHub Pages or Carrd): home ('AI PM for payments & FS'), 5 project pages. Each: problem → approach → architecture → evals & results → economics → learnings, with the demo video.

Session	What to do: step by step
S2 3 h	• Resume rewrite: headline 'Product Manager, AI & Payments'; an 'AI Product Projects' section with metric-led bullets (e.g., 'Built RAG copilot over 35 RBI/NPCI documents; 92% answer accuracy on 40-question golden set at ₹0.30/query'). Mirror JD keywords from your tracker; check ATS parsing. • Reframe consulting experience in PM language: discovery, prioritisation, stakeholder alignment, GTM, pricing.
S3 2 h	• LinkedIn: headline, About, Featured (projects), skills. Schedule 3 posts for the next 3 weeks.
S4 3 h	• Write the capstone case study (1,500 words), the piece you'll send hiring managers.
S5 2 h	• Ask 3 working AI PMs for 20-minute feedback on your portfolio (outreach template in Appendix F).

Deliverable: Live portfolio site, AI-PM resume, updated LinkedIn, capstone case study.

Self-check (answer out loud, without notes):

- What's your 30-second pitch?
- Which project do you lead with for a Razorpay role vs a Mastercard role?

Week 23 · Interview preparation across all rounds

Goal	Rehearse every round type until your answers are structured, specific and grounded in your own projects.
Why (JD evidence)	Loops are ~35% behavioural, ~20% AI product sense, ~15% technical, ~15% execution, ~15% strategy/presentation (Part B6).
Time	14–16 hours

Session	What to do: step by step
S1 3 h	• Behavioural: write 8 STAR stories (leadership, conflict, failure, ambiguity, influence, data-driven decision, customer obsession, 2 AI-specific stories from your projects: a failure mode you caught, a trade-off you made). Practise out loud.
S2 3 h	• AI product sense: 4 cases using the structure in Appendix E (users → problem → AI-fit → solution & autonomy → UX for uncertainty → evals & metrics → risks → launch).
S3 3 h	• Technical depth: drill the technical question bank in Appendix E until each answer is under 2 minutes. Have Claude play a sceptical staff engineer.
S4 3 h	• Execution & metrics: 4 cases (metric drop; north star for an AI feature; hallucination diagnosis; A/B test that shows higher engagement but more complaints). • Live prototyping drill: build a working prototype of a random prompt in 30 minutes with an AI app builder (for Meta-style rounds).
S5 2–3 h	• Strategy & take-home: practise a 48-hour take-home (AI PRD + 5-slide presentation). Do 1 full mock loop with a peer (or on Aced/Exponent peer mocks).

Deliverable: 8 STAR stories, 12 recorded case answers, 1 take-home, 1 full mock loop with feedback.

Self-check (answer out loud, without notes):

- Give your 'AI failure' story in 2 minutes.
- RAG vs fine-tuning for a bank's policy bot: answer in 90 seconds.
- Define a north star for Claude Code or ChatGPT, and 3 guardrails.

Week 24 · Applications, networking and the next 12 weeks

Goal	Build a steady application pipeline, convert your network into referrals, and set up continuous learning.
Why (JD evidence)	76% of companies hiring AI PMs posted just one role, so the market is wide. Referrals and tailored applications beat volume.
Time	12–15 hours

Session	What to do: step by step
S1 3 h	• Build a target list of 40 roles: 10 reach (labs, big tech), 20 target (Indian fintech/product companies, FS majors, AI startups), 10 safer (AI PM roles at consultancies, GCCs of global banks). Use this repo's job tracker + LinkedIn + career pages. Update the JD tracker.
S2 3 h	• Outreach: 20 referral requests (alumni, ex-colleagues, AI PM community members). Personalise with a relevant project link.
S3 3 h	• Apply to 10 roles with tailored resumes and a 3-sentence 'why me for this role' note linking the most relevant project.
S4 2–3 h	• 2 more mock interviews (peers or mentors). Fix your two weakest answers.
S5 2 h	• Retro: re-score C1–C10 against your Week 0 baseline. Plan the next 12 weeks: 10 applications and 5 conversations a week, 1 new mini-project every 2 weeks, 1 post a week, weekly reading (Appendix D).

Deliverable: 40-role pipeline, 20 outreach messages sent, 10 tailored applications, retro with new scores.

Self-check (answer out loud, without notes):

- Which 5 roles are your best fit, and what's your specific angle for each?
- What will you build next to stay current?

Final gate: are you ready?

- **Portfolio**: 2 teardowns · RAG copilot · agent · eval & red-team report · PRD + business case · capstone + case study.
- **Competencies**: Level 3 in C2–C5 and C7–C10; Level 2+ in C1 and C6.
- **Interviews**: you can do a 35-minute AI product-sense case, answer the technical bank crisply, and tell 8 stories without notes.
- If not, repeat the weakest phase's final week before applying at volume. Keep applying to 'reach' roles regardless.

Appendices

Appendix A · Glossary

Term	Plain-English meaning
A/B test	Randomly split users between versions to measure causal impact on metrics.
A2A (agent-to-agent)	Protocols that let agents built by different companies communicate and delegate tasks.
Agent	A model running in a loop that chooses its own next actions (tool calls) to reach a goal.
Agentic commerce	Agents discovering, buying and paying on a user's behalf under a mandate (card-network agent tokens, UPI pilots, checkout protocols).
Alignment	Making model behaviour match human intentions and values (helpful, honest, harmless).
Attention	The transformer mechanism that lets each token weigh every other token in context.
Base model	A pre-trained model before instruction tuning; an autocomplete engine.
Benchmark	A standard public test set (MMLU, GPQA, SWE-bench...) for comparing models; not the same as your product evals.
BM25	Classic keyword-search ranking algorithm; combined with vectors in 'hybrid search'.
Chunking	Splitting documents into passages for retrieval.
Context engineering	Deciding what information goes into the model's context window, in what form and order.
Context window	Maximum tokens a model can take in at once (input + output).
Copilot	AI that assists a human who stays in control (drafts, suggests); contrasted with autonomous agents.
Distillation	Training a small model to imitate a larger one, cutting cost and latency.
DPO / RLHF / RLAIF	Post-training methods that tune models on preferences from humans (RLHF) or AI graders (RLAIF); DPO is a simpler variant.
Drift	Model or data behaviour changing over time so quality degrades.
Embedding	A vector of numbers representing meaning; similar meanings are close together.
Error analysis	Reading real outputs, labelling failures, counting failure types, fixing the most common.
Eval (evaluation)	A repeatable test of AI quality: dataset + grader + metric.
Few-shot prompting	Including a few worked examples in the prompt.
Fine-tuning	Further training a model on your examples to specialise its behaviour.
Foundation model	A large general model (e.g., Claude, GPT, Gemini, Llama) adapted to many tasks.
Function / tool calling	The model returns a structured request to run a named tool; your code executes it.
Golden dataset	Curated test cases with expected behaviour, used for regression evals.
Grounding	Tying answers to provided sources (documents, data) rather than model memory.
Guardrails	Checks and limits around a model: input filters, output checks, action limits.
Hallucination	Fluent but false or unsupported output.

Term	Plain-English meaning
HITL (human-in-the-loop)	A human reviews or approves some AI outputs or actions.
Hybrid search	Combining vector (semantic) and keyword search.
Inference	Running a trained model to produce outputs.
Jailbreak	A prompt that tricks a model into breaking its safety rules.
Latency (TTFT, p95)	Response time. TTFT = time to first token; p95 = the 95th-percentile response time.
LLM	Large language model: a transformer trained on large text corpora to predict tokens.
LLM-as-judge	Using a model to grade outputs against a rubric; must be calibrated with human labels.
LLMOps / MLOps	Practices and tooling for deploying, monitoring and updating models in production.
MCP (Model Context Protocol)	Open standard for connecting AI apps to tools and data through 'MCP servers'.
Model card / system card	Documentation of a model's intended use, evaluations, limits and safety testing.
Model router	Logic that sends each request to the most suitable (often cheapest adequate) model.
Multimodal	Handles several input/output types: text, images, audio, video.
Observability / tracing	Recording each step of an AI request (prompts, tool calls, tokens, latency) for debugging.
Open-weight model	A model whose weights are published so you can host and modify it (e.g., Llama, Qwen).
Orchestration	The code that coordinates prompts, tools, retrieval and models into a workflow.
pass@k	Probability that at least one of k attempts succeeds; used for agent and code evals.
Post-training	Everything after pre-training: SFT, preference tuning, RL, safety tuning.
Pre-training	Training on a huge corpus to predict the next token.
Precision / recall	Of items flagged, % correct (precision); of true items, % caught (recall).
Prompt caching	Reusing processed prompt prefixes to cut cost and latency on repeated context.
Prompt injection	Malicious instructions in user input or retrieved content that hijack the model.
Quantisation	Storing weights at lower precision to run models cheaper and faster.
RAG	Retrieval-Augmented Generation: fetch relevant documents, then generate a grounded answer.
Reasoning model	A model trained to 'think' through intermediate steps before answering; slower, better on hard tasks.
Red-teaming	Deliberately attacking a system to find safety and security failures.
Reranker	A model that re-orders retrieved passages by relevance.
Rubric	Explicit criteria defining a good output, used by human or LLM graders.
SFT	Supervised fine-tuning on instruction → answer pairs.
Structured output	Forcing the model to return valid JSON matching a schema.
System prompt	Standing instructions that define the model's role, rules and style in a product.
Temperature	Sampling setting controlling randomness of outputs.

Term	Plain-English meaning
Token	Unit of text a model reads and writes (~¾ of an English word).
Transformer	Neural-network architecture behind modern LLMs, built on attention.
Vector database	Store optimised for similarity search over embeddings (e.g., pgvector, Pinecone, Chroma).
Vibe coding	Building software by describing it to an AI coding tool and iterating on the result.

Appendix B · 24-week tracker

Print this page or copy it into your learning log. Tick each week only when the deliverable is published (not just done).

Week	Deliverable	Done ☐	Date
0	Portfolio repo, baseline scores, JD tracker (10 JDs), post #0	☐	
1	Confusion-matrix cost sheet, ML cheat sheet, post #1	☐	
2	Token-cost table, 'How ChatGPT works' explainer, post #2	☐	
3	Model landscape sheet, bake-off, model selection memo	☐	
4	Two AI product teardowns (Artefact #1) · Phase 1 gate	☐	
5	System prompt v1–v3 + changelog, 30-case test sheet, JSON accuracy	☐	
6	Batch-classifier notebook (accuracy + cost), 5 SQL queries	☐	
7	RAG notebook, 40-question golden set, experiments, design doc	☐	
8	Deployed Payments Policy Copilot + README + demo (Project #2) · Phase 2 gate	☐	
9	2-tool agent, scored use-case table, chosen agent	☐	
10	MCP server, n8n/Zapier workflow, threat model	☐	
11	Dispute agent with tracing, 30-scenario results	☐	
12	Hardened agent, before/after table, demo (Project #3) · Phase 3 gate	☐	
13	Error-analysis Pareto, 100-case golden set, quality bar	☐	
14	3 calibrated judges, automated eval run, dashboard, A/B plan	☐	
15	Red-team report, guardrails, incident playbook	☐	
16	Eval & red-team report (Project #4), RAI checklist, system card · Phase 4 gate	☐	
17	5 interviews, workflow map, opportunity memo	☐	
18	AI PRD with metrics tree and rollout plan	☐	
19	Cost model, pricing, GTM, business case (Project #5)	☐	
20	Exec update, risk memo, 3 recorded cases · Phase 5 gate	☐	
21	Capstone MVP, eval report, demo	☐	
22	Portfolio site, AI-PM resume, LinkedIn, case study	☐	

Week	Deliverable	Done ☐	Date
23	8 STAR stories, 12 case answers, take-home, mock loop	☐	
24	40-role pipeline, 20 outreach, 10 applications, retro · Final gate	☐	

Appendix C · AI PRD template

Use this for Weeks 11, 18 and interview take-homes. Sections marked **(AI)** don't exist in a classic PRD.

Section	What to write
1. Summary	One paragraph: the problem, for whom, the proposed AI solution, the expected outcome.
2. Problem & evidence	User quotes, data (volumes, time spent, error rates, cost), why now.
3. Users & jobs-to-be-done	Primary/secondary users, their workflow today (map), where it hurts.
4. Goals & non-goals	What success looks like; what's explicitly out of scope.
5. Solution overview	User experience, AI pattern (copilot / workflow / agent), autonomy level (L1–L5) and why.
6. (AI) Behaviour spec	MUST do · MUST NOT do · ESCALATE when... Tone and format. 10–20 example inputs with ideal outputs.
7. (AI) Data & context	Knowledge sources (RAG), tools/APIs (MCP), user data used, permissions, freshness, consent (DPDP).
8. (AI) Model strategy	Model choice(s), routing, fallback provider, fine-tuning (if any), data residency.
9. (AI) Eval plan & ship gate	Rubric, golden dataset (size and mix), graders (code / LLM judge / human), thresholds to ship, regression process.
10. Metrics	North star, input metrics, AI-quality metrics, guardrail metrics, business metrics. Instrumentation plan.
11. (AI) Risks & guardrails	Failure modes (hallucination, injection, bias, leakage, excessive agency), mitigations per layer (Figure 12), red-team plan, incident playbook.
12. (AI) Cost & latency budget	Cost per task target, p95 latency target, volume forecast, margin impact.
13. UX for uncertainty	Citations, confidence signals, undo, approvals, feedback, handoff to human, onboarding and expectations.
14. Rollout plan	Dog-food → 1% → 10% → 50% → GA; criteria per stage; A/B design; rollback triggers; comms and support readiness.
15. Dependencies & team	Engineering, ML, design, data, legal/risk, GTM; RACI.
16. Open questions	What we don't know yet and how we'll find out (spikes, prototypes).

Appendix D · Resource library

Everything referenced in Part C, grouped by theme. Titles are as of this writing; check for newer editions. Where a resource has moved, search the exact title.

Resource	Type	Cost	Weeks
Foundations			
Google Machine Learning Crash Course	Course (web)	Free	1
AI for Everyone / Generative AI for Everyone (Andrew Ng)	Course	Free to audit	1
3Blue1Brown: Neural Networks series (chapters 1–2, 5–7)	Video	Free	2
Andrej Karpathy: Intro to LLMs; Deep Dive into LLMs like ChatGPT; How I use LLMs	Video	Free	2–3
Jay Alammar: The Illustrated Transformer / Word2vec	Article	Free	2
Artificial Analysis · LMArena (model comparisons)	Web tools	Free	3
Building with LLMs			
Anthropic prompt-engineering interactive tutorial and 'courses' repo (GitHub); Anthropic Academy	Course	Free	5, 9–10
Anthropic: 'Effective context engineering for AI agents', 'Introducing Contextual Retrieval'	Article	Free	5, 7
Python for Everybody (py4e.com) · Kaggle Learn Python / SQL	Course	Free	6
SQLBolt · Mode SQL tutorial	Interactive	Free	6
DeepLearning.AI short courses: LangChain: Chat with Your Data; Building & Evaluating Advanced RAG	Course	Free	7
NotebookLM · Streamlit · Hugging Face Spaces · Lovable / Bolt / v0 / Replit	Tools	Free tiers	7–8
Claude Code · Cursor	AI coding tools	Paid (trial/free tiers vary)	6–12, 21
Agents & MCP			
Anthropic: 'Building effective agents'	Article	Free	9
OpenAI: 'A practical guide to building agents'	Guide (PDF)	Free	9
DeepLearning.AI: Agentic AI (Andrew Ng); AI Agents in LangGraph; MCP: Build Rich-Context AI Apps with Anthropic	Course	Free	9–10
modelcontextprotocol.io (spec, SDKs, example servers)	Docs	Free	10
OWASP Top 10 for LLM Applications · Simon Willison on prompt injection	Security	Free	10, 15
n8n · Zapier · Langfuse / LangSmith / Arize Phoenix	Tools	Free tiers	10–11
Evals & responsible AI			
Hamel Husain: 'Your AI Product Needs Evals' + blog · Eugene Yan on LLM evals	Articles	Free	13
Lenny's Newsletter: 'Beyond vibe checks: A PM's complete guide to evals'	Article	Free/paid	13
Anthropic: 'Demystifying evals for AI agents'	Article	Free	13

Resource	Type	Cost	Weeks
AI Evals for Engineers & PMs (Hamel Husain & Shreya Shankar, Maven)	Cohort course	Paid (optional)	13–14
Promptfoo · Braintrust · Microsoft Presidio	Tools	Free / free tiers	14–15
Kohavi, Tang & Xu: Trustworthy Online Controlled Experiments	Book	Paid	14
DeepLearning.AI: Red Teaming LLM Applications; Quality and Safety for LLM Applications	Course	Free	15
DPDP Act 2023 & Rules · RBI FREE-AI report · MeitY AI Governance Guidelines · EU AI Act · NIST AI RMF	Primary sources	Free	16
Product craft & career			
Google PAIR People + AI Guidebook · Microsoft HAX Guidelines	Guide	Free	4, 12
Chip Huyen: AI Engineering (O'Reilly, 2025)	Book	Paid	17 (reference throughout)
Marily Nika: Building AI-Powered Products (O'Reilly)	Book	Paid	17–20
Teresa Torres: Continuous Discovery Habits · Rob Fitzpatrick: The Mom Test	Books	Paid	17
Marty Cagan: Inspired · Shreyas Doshi's frameworks	Book / posts	Paid / free	20
Ethan Mollick: Co-Intelligence + 'One Useful Thing' newsletter	Book / newsletter	Paid / free	Any
Lenny's Podcast (Kevin Weil, Mike Krieger and other AI PM episodes)	Podcast	Free	4, ongoing
Aced (formerly Exponent) · IGotAnOffer · ai-pm-interview-prep (GitHub, 131 questions)	Interview prep	Paid / free	23
AI Product Manager Hiring Report 2026 (dataset used in Part B)	Data	Free (CC BY 4.0)	0, 22, 24

Keep up to date (15 minutes a day, after Week 24)

- Newsletters: Lenny's Newsletter, One Useful Thing (Ethan Mollick), Simon Willison's blog, Latent Space, The Batch (DeepLearning.AI), Aakash Gupta's Product Growth.
- Company engineering blogs: Anthropic, OpenAI, Google DeepMind, plus Indian builders (Razorpay, PhonePe, Swiggy, Sarvam).
- Model releases: read the system card and pricing page of every major release; re-run your capstone evals on it.
- India FS: RBI press releases and circulars, NPCI circulars, MeitY announcements.

Appendix E · Interview preparation kit

AI product-sense answer structure (35 minutes)

1. **Clarify** (2 min): goal, user segment, constraints (region, regulation, platform).
2. **Users & problem** (5 min): segments, pick one, their workflow and top pain points.
3. **AI-fit** (3 min): why AI beats rules/UX here; cost of error; autonomy level.
4. **Solution** (8 min): 2–3 options; pick one; key flows; AI pattern (Figure 8); what the model needs (context, tools).
5. **UX for uncertainty** (3 min): citations, confidence, undo, approvals, feedback, human handoff.
6. **Evals & metrics** (6 min): rubric, golden set, ship gate; north star, guardrails; A/B plan.

7. **Risks** (4 min): hallucination, injection, bias, privacy (DPDP), cost; mitigations.

8. **Launch & iterate** (4 min): staged rollout, what you'd learn first, v2.

Technical questions: with what a strong answer covers

Question	A strong answer covers
Explain how an LLM works to a non-technical executive.	Next-token prediction; pre-training on huge text; post-training to be helpful and safe; it predicts plausible text rather than looking things up, hence hallucinations.
When would you use RAG vs fine-tuning vs prompting?	The ladder (Figure 9): prompt first; RAG for knowledge and freshness; fine-tune for consistent style/format or cost via distillation; measure each step with evals.
Users say the bot is 'confident but wrong'. Diagnose it.	Pull traces → is it retrieval (right doc not found) or generation (doc found, misused)? Check freshness, chunking, prompt, model; add groundedness check and 'I don't know' behaviour; add cases to golden set.
How do tokens and context windows affect cost and UX?	Cost = tokens × price; long context costs more and is slower; Indic languages use more tokens; curate context; caching; streaming.
What is an eval? Design one for a bank chatbot.	Dataset + grader + metric; rubric (correctness, groundedness, PII, escalation, tone); code checks + calibrated LLM judge + human sampling; ship gate; online metrics.
How do you know an LLM judge is reliable?	Hand-label a sample; measure agreement and TPR/TNR; iterate the prompt; re-check periodically; binary criteria; watch for verbosity/position bias.
What is prompt injection? How do you defend?	Direct vs indirect; untrusted content; least-privilege tools; approvals for actions; input classifiers; break the lethal trifecta.
What's an agent vs a workflow? When would you avoid an agent?	Model-directed loop vs predefined steps; avoid when steps are known, errors are costly, or latency/cost budgets are tight; start with the simplest pattern.
Explain MCP.	Open standard for connecting AI apps to tools and data; build a server once, use it from any host; security considerations.
Precision vs recall for fraud or content moderation?	Define both; trade-off via threshold; choose by the cost of false positives vs false negatives; show a confusion matrix.
How would you reduce latency for a voice assistant?	Smaller/faster model, streaming, shorter prompts, caching, parallel tool calls, speculative responses, regional hosting.
How do you choose between models?	Your own eval results on your task; cost per task; latency; context/modality needs; data residency; reliability; fallback plan.
What changes when a new model version ships?	Treat as a product change: run full eval suite, compare quality/cost/latency, check regressions and behaviour changes, staged rollout.
How do you measure an agent's performance?	Task success rate, pass@k, steps per task, cost per task, escalation rate, unsafe-action attempts, time to completion; trace-level review.
What's the data flywheel for your product?	Usage → traces and feedback → better evals, prompts, retrieval, fine-tunes → better product → more usage. Mind consent and privacy.

Product sense, execution and strategy prompts to practise

- Design an AI assistant for small merchants on a UPI payments app.
- Design an AI feature to reduce failed-payment support tickets by 30%.

- Improve a bank's loan-application journey with AI without increasing risk.
- Build an AI copilot for credit-card dispute analysts.
- Design an agent that books travel and pays for it: what autonomy, what guardrails?
- North-star metric for Claude Code / ChatGPT / Perplexity, and three guardrails.
- Your AI support bot's resolution rate rose 10% but CSAT fell. What happened?
- Cost per conversation doubled after a model upgrade. What do you do?
- Should a mid-size Indian bank build or buy its customer-service AI?
- Is your AI product defensible if a frontier lab launches the same feature?
- How would you price an AI agent that resolves customer disputes?
- An exec wants to 'add AI' to every product this quarter. How do you respond?

Behavioural stories to prepare (STAR format)

- A time you led without authority (from consulting: aligning client stakeholders).
- A decision you made with incomplete data.
- A failure and what you changed afterwards.
- **AI-specific:** a failure mode you found in your own AI project and how you fixed it.
- **AI-specific:** a trade-off between quality, cost and speed you made, with numbers.
- A conflict with an engineer or senior stakeholder.
- A time you said no to a good idea.
- A time customer insight changed your plan.

Appendix F · Outreach templates

Referral request (LinkedIn / email, under 120 words)

Hi [Name], I'm [one line: MBA / consulting background] moving into AI product management with a focus on payments. I noticed [Company] is hiring a [Role]. I recently built [project, one line + a metric, e.g., 'a dispute-resolution agent that resolves 70% of test cases with human approval for refunds'] ([link]). Would you be open to referring me, or to a 15-minute chat about the team? Happy to share a one-page summary. Thank you!

Portfolio feedback request to a working AI PM

Hi [Name], I've followed your posts on [topic]. I'm six months into a structured switch to AI PM and have built [2 projects]. Could I get 20 minutes of blunt feedback on my portfolio ([link])? I'd especially value your view on [specific question: e.g., whether my eval approach would pass your team's bar]. I'll come with specific questions and respect your time.

Appendix G · Sources

Dataset

- Ashtekar, A. (2026). **AI Product Manager Hiring Report 2026: What 2,448 Job Postings Reveal.** Dataset: github.com/AbhishekAshtekarDev/ai-product-manager-hiring-report-2026 (CC BY 4.0). All Part B charts are built from this data.

Job descriptions (read via search extracts, September 2026)

- Anthropic: Product Manager, Claude Code Model Performance; Product Manager, Consumer; Product Manager, Core Product (Claude.ai) (job-boards.greenhouse.io/anthropic).
- OpenAI: Product Manager, Core Models; Product Manager, API Agents; Product Manager, API Infrastructure (openai.com/careers).

- Google DeepMind: Sr PM Gemini Post Training; PM Gemini API Models; PM Gemini App (google.com/about/careers).
- Microsoft AI: AI Product Management, Copilot; Principal PM roles (microsoft.ai/job); Microsoft IDC Sr PM, Business & Industry Copilots.
- Amazon: Sr. PM-Technical, Amazon Bedrock; Sr. PM-Technical, Agentic AI, AWS Transform (amazon.jobs).
- Meta: Product Manager, Central Product Responsibilities; Product Manager (Leadership) (metacareers.com).
- Glean (PM, AI Quality); Perplexity (PM, Growth / SMB); Sierra (PM, Agent Development); Decagon (Senior Agent PM); Intercom (Senior / Staff AI PM); Figma (PM, AI Platform); Notion (PM, AI Products); Adobe (Principal PM, Firefly); Salesforce (PM, Agentforce); Atlassian (Principal PM, Rovo Search & AI Quality).
- Uber (PM II, Conversational AI Platform; Sr PM, AI Quality, Bengaluru); Spotify (PM, AI Observability/Evaluation Platform); ASAPP (Sr PM, Observability Tools); HighLevel (Group PM, AI); Mindtickle (Group PM); TTEC Digital (PM, Services).
- Razorpay (PM II, AI); PhonePe (AI Product Manager, ML); Paytm (PM, Lending AI & Platform; Sr PM, AI Agents); Swiggy (PM I); Sarvam AI (PM; PM Models; PM Model APIs); Freshworks (Sr / Staff PM, Freddy AI); Flipkart (PM); Walmart (Sr PM, Gen-AI).
- Mastercard (Lead Agentic AI PM; Manager, PM, Agentic Commerce); Visa (Sr PM, Agentic Commerce); JPMorgan Chase (AI/ML PM VP; AI PM, AI Agents); Capital One (PM, Enterprise AI/ML); American Express (Sr PM, ML & Personalization); Stripe (Staff PM, ML Foundations & GenAI; AI PM, Professional Services).
- Accenture (AI Product Management Practitioner); BCG X (AI Product Manager); McKinsey QuantumBlack (Senior Product Analyst).

Market, salary and interview sources

- productleadership.com: 'AI Product Manager Salary in India' (2026); Glassdoor India salary data as cited there.
- axiasearch.com: 'AI Product Management Jobs in 2026: What 12,400 Postings Reveal'.
- Interview guides: Northeastern University career design, 'AI Product Manager Interview Questions (2026 Guide)'; Aced (formerly Exponent); IGotAnOffer; KORE1; github.com/landedjobs/ai-pm-interview-prep.
- Engineering and product writing cited in Part A and C: Anthropic ('Building effective agents', 'Demystifying evals for AI agents', 'Effective context engineering for AI agents', 'Introducing Contextual Retrieval'); OpenAI ('A practical guide to building agents', Model Spec); Hamel Husain ('Your AI Product Needs Evals'); OWASP Top 10 for LLM Applications.

A note on currency

AI changes monthly. Model names, prices and some regulations in this guide will date quickly. The **concepts, flows and learning sequence** will not. When a fact matters for a decision or an interview, check the primary source.